Tunisian Republic
Ministry of Higher Education and Scientific Research
Directorate General of Technological Studies
Higher Institute of Technological Studies of Sfax
Department: Information Technology

MANDATORY END-OF-STUDIES INTERNSHIP REPORT

With a view to obtaining the National Diploma of

Licence in Information Technology

Option : Information Systems Development

Title

Collectra : MVP SaaS of Debt Collection

Developed by

Taha Bchir

Presented on 23/05/2026 to the jury committee

M.	**Prénom et NOM**	**President**
Mme	**Prénom et NOM**	**Rapporteur**
M.	**Prénom et NOM**	**Supervisor (ISET)**
M.	**Prénom et NOM**	**Supervisor (ISET)**

University Year 2025 / 2026

Code :

Acknowledgements

I would like to express my sincere gratitude to all those who supported me throughout this internship and the preparation of this report.

First, I would like to thank **CyberBridge** for welcoming me and offering the opportunity to work on a real production project. I am especially grateful to *[company tutor name and title]* for the guidance, availability, and valuable feedback throughout the internship period.

I would also like to thank my academic supervisor, *[supervisor name and title]*, for the continuous support, constructive advice, and time devoted to reviewing this work.

Finally, I express my gratitude to the faculty of *[institution name]* and to my colleagues for their encouragement and support.

Abstract

This report presents the final-year internship project carried out at **CyberBridge** under the title **Collectra** – a web-based platform for debt collection management and customer reminder campaigns.

Debt collection teams commonly rely on fragmented tools: spreadsheets for data storage, email threads for communication history, and general-purpose Customer Relationship Management systems that provide no lifecycle tracking for outstanding debts. Collectra addresses these gaps by bringing debt records, campaigns, team actions, and customer communication into a single, workspace-based Software as a Service environment.

The platform delivers secure JSON Web Token-based authentication with Hypertext Transfer Protocol-only session cookies, strict workspace-level data isolation, role-based access control for managers and agents, a complete debt lifecycle from import through payment, time-limited secure personal customer links, Stripe-backed payment confirmation with idempotent verification, automated invoice generation and delivery via Brevo, campaign-level email tracking analytics, per-debt multi-currency Stripe payments, an operational dashboard for managers, and an in-app support chatbot.

Development followed the Scrum agile methodology across four two-week sprints, with task tracking managed in Linear. The application is deployed on **Vercel** under the domain `collectra.xyz`, using a GitHub-based Continuous Integration and Continuous Deployment pipeline where changes are merged from the `develop` branch into `production` and published automatically. The technical stack is built on Next.js, React, TypeScript, Hono, PostgreSQL (Supabase), and Prisma Object-Relational Mapper, organized in a Turborepo monorepo.

Keywords: debt collection, Software as a Service, campaign management, JSON Web Token, Scrum, Next.js, TypeScript, PostgreSQL, Prisma, Supabase, Stripe, invoice generation, Vercel, Continuous Integration and Continuous Deployment, email tracking.

Contents

List of Figures

List of Tables

General Introduction

Debt collection is a critical business process for financial institutions, telecom operators, and service companies that manage large portfolios of unpaid invoices. Despite its importance, many collection teams still rely on fragmented tools: spreadsheets for data storage, email threads for communication history, and separate Customer Relationship Management systems that were never designed for debt lifecycle tracking. The consequences are predictable: poor traceability, weak team collaboration, and no single operational view of campaign progress.

The **Collectra** project was developed during this internship at CyberBridge specifically to address these gaps. Collectra is a Software as a Service web platform that brings debt data, campaigns, team activities, and customer communication into one secure, workspace-based environment. It replaces scattered tools with a single, role-aware application that supports the full debt lifecycle, from initial import to payment confirmation and invoice delivery.

The internship followed professional practices: version control with Git, code review through pull requests, task tracking in Linear, and automated deployments to a production domain. Each increment was demonstrated to CyberBridge stakeholders before being merged, which reduced the risk of building features that did not match real collection workflows.

This report is organized as follows:

- **Chapter 1 – Company Presentation and Project Context:** Host company, problem statement, analysis of existing solutions, and proposed solution.
- **Chapter 2 – Requirements and Specifications:** Actors, functional and non-functional requirements, Scrum planning, quality assurance, security architecture, general architecture, deployment infrastructure, and end-to-end user journeys.
- **Chapter 3 – Sprint 1: Core Authentication and Workspace Setup:** Authentication system, workspace management, and member invitation flow.
- **Chapter 4 – Sprint 2: Campaign and Debt Management:** Campaign lifecycle, Comma-Separated Values import, debt status tracking, and customer personal links.
- **Chapter 5 – Sprint 3: Collaboration and Reporting:** Action history, payment promise recording, and the operational manager dashboard.
- **Chapter 6 – Sprint 4: Payment, Invoicing, and Analytics Integration:** Payment confirmation, multi-currency Stripe checkout, invoice generation, email tracking, and final analytics integration.

- **Chapter 7 – Cross-Cutting Technical Topics:** Email pipeline, payment flow, in-app support chatbot, error handling, and platform-wide engineering practices.
- **Appendices:** Domain glossary, interface walkthrough with screenshots, Application Programming Interface reference, and database schema.

1 Company Presentation and Project Context

Introduction

This chapter presents the host company, the project context, the problem statement, an analysis of existing solutions, and the proposed Collectra platform.

1.1 Presentation of the Host Company

The internship was carried out at **CyberBridge** , a software company founded in 2019 in Wuppertal, Germany, by experienced IT professionals. CyberBridge builds tailored IT solutions and positions itself as a strategic partner from the initial concept through to successful product delivery. With certified expertise and broad industry experience, the company focuses on improving product quality and optimizing business processes for its clients.

CyberBridge	**CyberBridge** — software company founded in 2019 in Wuppertal, Germany. Strategic partner from concept to product delivery, with certified expertise across industries.

Table 1.1: CyberBridge – Host Company

CyberBridge offers the following core services:

- **Project Management:** structured delivery with planning tailored to each customer's needs;
- **Enterprise Software Development:** modern, scalable systems that support business growth;
- **Blockchain and AI:** enterprise-grade solutions for automation, transparency, and secure digital transformation;
- **Requirements Engineering:** expert consulting to define software requirements clearly and completely;
- **Software Quality:** testing and QA support following an International Software Testing Qualifications Board-certified process;
- **UI/UX Design:** usability-focused design for both internal teams and end users.

During the internship, CyberBridge provided a real production-like development context, direct feedback from stakeholders, and access to team workflows and sprint planning in Linear.

1.2 Project Context

1.2.1 Main Issue

Debt collection teams face a recurring set of operational challenges. Follow-up work is often fragmented across multiple tools: debt data is stored in spreadsheets, contact history is tracked in email threads, and reminder statuses are maintained manually. This makes it difficult to audit past actions, share information across team members, or monitor campaign progress in real time.

The specific problems identified at the start of the project were:

- scattered debt data without a single source of truth;
- weak traceability of reminders, promises, and payments;
- limited collaboration between agents and managers;
- no unified view of campaign progress or debt lifecycle status;
- no secure way for customers to consult their own debt information.

The central question is therefore: *how can we build a secure, scalable platform that centralizes debt collection work while keeping data strictly isolated per workspace?*

1.2.2 Stakeholder Expectations

During requirements workshops at CyberBridge, three expectations guided the product direction. **Managers** needed a reliable overview of campaign progress, team activity, and payment outcomes without exporting spreadsheets. **Agents** needed fast daily operations: importing debts, updating statuses, generating customer links, and recording promises with minimal friction. **Customers** needed a simple, trustworthy page to understand their debt and complete payment without creating an account. These expectations translated directly into the sprint backlog and into non-functional requirements (security, traceability, and performance under bulk import).

1.2.3 Analysis of Existing Solutions

A study of available tools revealed the following limitations:

- **Spreadsheets (Excel, Google Sheets):** flexible but difficult to audit, with no role governance, no lifecycle tracking, and no secure customer access;
- **Generic Customer Relationship Management tools (Salesforce, HubSpot):** designed for sales pipelines, not debt status lifecycles; often too complex or costly for small collection teams;
- **Manual link sharing :** no expiration control, no authentication, and no audit trail.

1.2.4 Overview of the Proposed Solution

The proposed solution is **Collectra**, a workspace-based Software as a Service platform that covers all five critical capabilities that existing tools only address partially, as shown in Table 1.2.

Collectra offers:

- secure authentication and role-based access control;
- workspace-level data isolation for multi-team environments;
- campaign creation and bulk Comma-Separated Values debt import;
- a full debt lifecycle (imported → notified → promised → paid / overdue);
- JSON Web Token-secured personal customer links with configurable expiration;
- an operational dashboard for managers.

Feature	Spreadsheets	Generic CRM*	Collectra
Debt lifecycle tracking	Partial	Partial	**Full**
Secure personal customer link	No	Limited	**Yes**
Workspace-level data isolation	No	Limited	**Yes**
Operational dashboard	Limited	Yes	**Yes**
Collection-team role governance	No	Limited	**Yes**

Table 1.2: Comparison of Existing Solutions and Collectra

* Generic Customer Relationship Management platforms (e.g. Salesforce, HubSpot).

1.2.5 Internship Timeline and Milestone Deliverables

The internship followed a four-sprint calendar aligned with CyberBridge planning rituals. Table 1.3 summarises the main milestones, review dates, and deliverables demonstrated to stakeholders. Each milestone ended with a Vercel preview deployment so non-technical reviewers could validate user journeys without installing the project locally.

Beyond feature delivery, each sprint produced **traceable artefacts**: Linear issues linked to pull requests, Postman collections for Application Programming Interface regression, and annotated screenshots used in this report appendix. This traceability was essential when explaining design trade-offs during the final presentation, because every user story could be mapped to a concrete route, database table, and user interface screen.

Conclusion

This chapter introduced CyberBridge , described the operational problem in debt collection, analysed the limits of existing tools, and presented Collectra as the proposed solution. The

Period	Focus	Demonstrated Outcome
Weeks 1–2	Environment setup, backlog refinement, architecture sketch	Monorepo bootstrap, Supabase project, first authentication prototype
Weeks 3–4	Sprint 1 review	Login, registration, workspace creation, member invitation with email acceptance
Weeks 5–6	Sprint 2 review	Campaign Create, Read, Update, Delete, Comma-Separated Values import, debt lifecycle, personal links
Weeks 7–8	Sprint 3 review	Action history, payment promises, manager dashboard, role-based access control hardening
Weeks 9–10	Sprint 4 review	Stripe checkout, idempotent verification, invoice access, Brevo tracking analytics
Weeks 11–12	Stabilisation and documentation	Production merge to `collectra.xyz`, OpenAPI publication, report and defense preparation

Table 1.3: Internship Timeline and Milestone Deliverables

central design challenge – strict workspace isolation combined with a flexible debt lifecycle – directly shaped the technical decisions described in the following chapters.

2 Requirements and Specifications

Introduction

This chapter presents the project actors and requirements, the work environment and technology stack, the Scrum methodology and project planning, the product backlog, quality assurance and security architecture, the general technical architecture, and the deployment infrastructure of Collectra.

2.1 Collectra Platform Requirements

2.1.1 Identification of the Actors

The Collectra platform involves four main actors:

- **Owner / Manager:** responsible for workspace setup, team invitations, campaign supervision, and dashboard monitoring;
- **Agent:** performs daily debt follow-up, records actions and promises, and generates customer personal links; agents may also create new workspaces and will be assigned the Owner role for any workspace they create by default;
- **Customer:** accesses personal debt information through a secure, time-limited link without needing an account;
- **System services:** the Supabase authentication provider and the email notification service (Brevo).

2.1.2 Functional Requirements

- Secure user authentication and session management;
- Workspace creation and strict workspace-based data isolation;
- Team invitation and role-based permission assignment;
- Campaign creation, configuration, and lifecycle management;
- Bulk Comma-Separated Values import for debt records;
- Debt status tracking across a defined lifecycle;
- Generation and management of JSON Web Token-secured customer personal links (48-hour expiration by default);

- Customer debt consultation via personal link (no account required);
- Stripe payment confirmation with idempotent verification;
- Invoice generation and delivery after confirmed payment;
- Dashboard indicators for campaign and debt monitoring;
- Action history log for full traceability;
- Campaign-level email tracking analytics;
- Per-debt currency selection for Comma-Separated Values import, debt records, and Stripe Checkout;
- In-app support chatbot for managers and agents (OpenRouter with rule-based fallback).

2.1.3 Non-Functional Requirements

- **Security:** JSON Web Tokens signed with Hash-based Message Authentication Code SHA-256, Hypertext Transfer Protocol-only cookies, and route-level middleware checks for every protected endpoint;
- **Maintainability:** modular monorepo architecture with shared packages and a single Prisma schema as the source of truth;
- **Performance:** efficient handling of bulk Comma-Separated Values imports with Prisma transaction batching to avoid database timeouts;
- **Reliability:** validated Application Programming Interface contracts via Zod, database-level constraints, and idempotent payment confirmation logic;
- **Usability:** clean dashboard workflow with a minimal learning curve, validated through iterative stakeholder feedback during each sprint review.

Table 2.1 translates these principles into measurable targets used during sprint reviews at CyberBridge.

Category	Target	Verification Method
Security	No cross-workspace data leak on forbidden routes	Postman tests T1-10, T2-08, role-based access control scenarios
Performance	Comma-Separated Values import of 3 000+ rows completes without timeout	Preview deployment timing logs and chunked insert metrics
Reliability	Duplicate payment verification does not double-charge	Sprint 4 idempotency tests and Stripe replay simulations
Usability	Core agent actions within three clicks from debt table	Heuristic review during sprint demos

Table 2.1: Non-Functional Requirements – Targets and Verification

2.2 Work Environment

2.2.1 Technology Stack

Table 2.2 summarises the technologies used in Collectra.

Layer	Technology	Role	Logo
Frontend	Next.js 14, React, TypeScript	Pages, components, routing	
Styling	Tailwind CSS	User Interface layout and design system	
Backend	Hono, TypeScript, Zod	Representational State Transfer Application Programming Interface, validation, OpenAPI	
Database	PostgreSQL via Supabase	Persistent data storage	
Object-Relational Mapper	Prisma	Schema management and queries	
Authentication	Supabase Auth	Sessions, JSON Web Token, email auth	
Payments	Stripe	Checkout sessions and webhooks	–
Email	Brevo	Transactional email and tracking	–
Build tools	pnpm, Turborepo	Monorepo management and builds	
Deployment	Vercel	Hosting and Continuous Integration and Continuous Deployment pipeline	–
Support assistant	OpenRouter (DeepSeek model)	In-app chatbot for managers and agents	–

Table 2.2: Collectra Technology Stack

2.2.2 Development Tools

Table 2.3 lists the main development and collaboration tools used during the internship.

2.3 Project Management with Scrum

2.3.1 Methodology Choice

The project follows the Scrum agile framework. Scrum was chosen because it supports iterative delivery, allows priorities to be adjusted based on stakeholder feedback, and produces working software at the end of each sprint. Two-week sprint cycles made it possible to validate each feature set before moving forward to the next one.

Tool	Role	Logo
Visual Studio Code	Main Integrated Development Environment for frontend and backend development	
Git / GitHub	Version control and collaborative code management	
Linear	Scrum task management, sprint planning, and issue tracking	
Postman / Swagger UI	Application Programming Interface testing and endpoint validation	
Supabase Studio	Database inspection and query testing	

Table 2.3: Development and Collaboration Tools

2.3.2 Scrum Roles

- **Product Owner:** defines, orders, and maintains the product backlog based on business needs;
- **Development Team:** implements backend and frontend features sprint by sprint;
- **Scrum Master:** facilitates Scrum ceremonies and removes blockers.

Tasks were tracked in **Linear** using statuses (*Backlog, In Progress, In Review, Done*), priority levels, assignees, and sprint milestones.

2.3.3 Product Backlog

The product backlog lists all user stories sorted by business impact, as shown in Tables 2.4 and 2.5.

ID	User Story	Period	Priority
US-01	As a manager or agent I want to create a workspace so that my team data is isolated.	Weeks 3–4	High
US-02	As a manager I want to invite members so that agents can collaborate in the same workspace.	Weeks 3–4	High
US-03	As an agent I want to import a Comma-Separated Values file so that campaign debts are created in batch.	Weeks 5–6	High
US-04	As an agent I want to view and update debt statuses so that follow-up stays up to date.	Weeks 5–6	High
US-05	As an agent I want to generate a customer personal link so that the customer can access debt details.	Weeks 5–6	High

Table 2.4: Product Backlog for Collectra (part 1)

ID	User Story	Period	Priority
US-06	As a customer I want to open a secure link so that I can view my debt without creating an account.	Weeks 5–6	Medium
US-07	As a manager I want to view dashboard indicators so that I can monitor collection progress.	Weeks 7–8	Medium
US-08	As a manager I want action history logs so that all operations are traceable.	Weeks 7–8	Medium
US-09	As an agent I want to record payment promises so that commitment dates are tracked.	Weeks 7–8	Medium
US-10	As a manager I want role-based permissions so that sensitive actions are protected.	Weeks 7–8	High

Table 2.5: Product Backlog for Collectra (part 2)

2.3.4 Sprint Overview

Table 2.6 summarises the four sprints and their main deliverables.

Sprint	Period	Focus	Main Deliverables
Sprint 1	Weeks 3–4	Authentication & Workspace Setup	Login, sign-up, workspace creation (manager or agent), member invitation
Sprint 2	Weeks 5–6	Campaign & Debt Management	Campaign Create, Read, Update, Delete, Comma-Separated Values import, debt lifecycle, customer personal links
Sprint 3	Weeks 7–8	Collaboration & Reporting	Action history, payment promises, manager dashboard, role-based access control
Sprint 4	Weeks 9–10	Payment, Invoicing & Analytics	Stripe verification, per-debt currency, idempotent payment, invoice generation, email tracking analytics

Table 2.6: Sprint Overview

2.3.5 Scrum Ceremonies and Delivery Rhythm

Collectra was delivered through four consecutive two-week sprints. Each sprint opened with **sprint planning**, where user stories were selected from the product backlog, estimated, and assigned in Linear. **Daily coordination** was kept lightweight: short updates on progress, blockers, and integration risks, especially when backend contracts and frontend screens had to evolve together. At the end of each sprint, a **sprint review** demonstrated working features to CyberBridge stakeholders, and a **retrospective** captured process improvements for the next iteration (test data preparation, preview deployments, and clearer acceptance criteria for customer-facing flows).

This rhythm made it possible to validate authentication, import performance, dashboard indicators, and payment confirmation incrementally instead of deferring integration risks to the final weeks of the internship.

2.4 Quality Assurance and Testing Strategy

Quality assurance was integrated into every sprint rather than postponed to the end of the internship. The approach combined **functional tests** (documented in each sprint chapter), **contract validation** with Zod on the backend, and **manual exploratory testing** on Vercel preview deployments before merging to the production branch.

2.4.1 Testing Levels

- **Unit-level checks:** validation helpers, status-transition guards, and payment verification utilities were exercised with representative inputs and edge cases;
- **Integration tests:** Application Programming Interface endpoints were verified with Postman collections and Swagger UI, focusing on authentication cookies, workspace isolation, and import summaries;
- **End-to-end flows:** login, workspace invitation, Comma-Separated Values import, customer personal link access, Stripe checkout, and invoice retrieval were replayed on preview environments after each sprint review;
- **Regression discipline:** critical paths from earlier sprints were re-tested when shared middleware or database schema changed.

2.4.2 Test Data and Acceptance Criteria

Each sprint defined explicit acceptance criteria in Linear (expected status codes, visible user interface states, and database side effects). For imports, tests included files with delimiter variations, duplicate emails, invalid amounts, and large batches to confirm chunked inserts remained stable. For payments, tests covered duplicate verification calls to confirm idempotent confirmation.

2.4.3 Debt Status Lifecycle Model

Collectra models each debt as a finite state machine. Statuses are stored in the database as enumerated values and validated on every update. Table 2.7 lists the main states, their business meaning, and typical transitions performed by agents or automated customer actions.

The backend rejects illegal jumps (for example, `Paid` → `Imported`) through a dedicated transition guard. This design prevents accidental data corruption when agents bulk-update rows or when webhooks arrive out of order. From a product perspective, managers can filter dashboards and exports by status, which makes operational meetings more concrete: teams discuss counts per state instead of ambiguous spreadsheet colours.

2.4.4 Defect Management

Tasks assigned by the supervisor were managed in Linear with priority levels and related details about the affected module (`apps/web`, `apps/api`, or shared packages). High-priority

Status	Meaning	Typical Next States
Imported	Debt created from Comma-Separated Values or manual entry; not yet communicated	Notified, Overdue
Notified	Customer received a link or email; collection is active	Promised, Paid, Overdue
Promised	Customer or agent recorded a commitment date	Paid, Overdue, Notified
Paid	Payment confirmed (Stripe or controlled demo flow)	Terminal state
Overdue	Due date passed without payment; escalated follow-up	Promised, Paid, Notified

Table 2.7: Debt Status Lifecycle – States and Transitions

issues were completed first before moving to lower-priority tasks, while less important improvements were deferred to the backlog with clear priority labels.

2.5 Security Architecture

Security was treated as a cross-cutting concern from Sprint 1 onward. Collectra combines authentication, authorization, and tenant isolation at multiple layers.

2.5.1 Authentication and Session Model

Users authenticate through Supabase Auth. The backend issues a signed JSON Web Token stored in a **Hypertext Transfer Protocol-only** session cookie so client-side scripts cannot read it. Protected routes verify the token on every request before executing business logic.

2.5.2 Workspace Isolation and Role-Based Access Control

All operational data is scoped to a `workspaceId`. Middleware rejects cross-tenant access attempts with `403 Forbidden` even when a user is authenticated. Managers and agents receive different permissions: sensitive actions (invitations, campaign deletion, dashboard access) require the manager role.

2.5.3 Customer-Facing Security

Personal links rely on signed JSON Web Tokens with expiration. Public endpoints never expose internal identifiers without verification. Payment confirmation uses Stripe session identifiers, database-level locks, and idempotent verification to prevent duplicate charges.

2.5.4 Secrets and Deployment Hygiene

Database credentials, JSON Web Token secrets, Stripe keys, and Brevo credentials are injected as environment variables in Vercel. No secret is committed to Git. Preview deployments use separate configuration so test keys cannot affect production data.

2.6 General Architecture of the Application

Collectra is structured as a **monorepo** managed with pnpm and Turborepo. It contains four main packages:

- `apps/web` – the Next.js frontend application;
- `apps/api` – the Hono backend Representational State Transfer Application Programming Interface;
- `packages/db` – the shared Prisma client and database schema;
- `packages/types` – shared TypeScript types and Zod validation schemas.

The logical architecture separates four main domains: authentication and session management, workspace and team management, campaign and debt management, and customer access. Each domain maps to a dedicated set of Application Programming Interface routes and frontend pages. All inter-domain communication passes through typed Zod-validated contracts, ensuring that the Application Programming Interface surface is consistent and that both frontend and backend always share the same type definitions.

2.6.1 Monorepo Structure and Code Organization

The monorepo is orchestrated with Turborepo task pipelines (`build`, `lint`, `dev`) so unchanged packages are skipped during continuous integration. Shared types in `packages/types` prevent drift between the Next.js client and the Hono server: when a request schema changes, TypeScript compilation fails until both sides are updated.

- **Frontend (`apps/web`):** route groups for authentication, workspace dashboards, campaign screens, and customer-facing public pages;
- **Backend (`apps/api`):** route modules grouped by domain (auth, workspaces, campaigns, debts, public customer access, payments);
- **Database (`packages/db`):** Prisma schema, migrations, and generated client;
- **Shared contracts (`packages/types`):** Zod schemas reused for validation and OpenAPI generation.

This layout reduced duplication and made refactors safer: a single schema change propagates to the Application Programming Interface contract, the database layer, and the user interface forms in one coordinated change set.

2.6.2 Data Persistence and Domain Entities

PostgreSQL stores all business entities with strict foreign keys between workspaces, campaigns, debts, promises, and history tables. Prisma enforces referential integrity and provides typed queries for dashboard aggregations and bulk imports. Customer-facing actions are persisted in history tables so managers can audit who changed a status, when a promise was recorded, and when a payment was confirmed.

2.7 Deployment and Continuous Integration / Continuous Deployment Pipeline

2.7.1 Hosting Infrastructure

Collectra is deployed on **Vercel**, a cloud platform optimized for Next.js applications, under the custom domain `collectra.xyz`. The Supabase-managed PostgreSQL database runs on Supabase's cloud infrastructure, and all environment variables (database connection strings, JSON Web Token secrets, Stripe keys, Brevo application programming interface key) are configured as Vercel project environment variables and are never committed to the repository.

2.7.2 Git Branching Strategy

The project uses a two-branch Git workflow on GitHub:

- **`develop`:** the active development branch where all feature branches are merged after code review. This branch reflects the latest integrated work-in-progress state.
- **`production`:** the stable branch that drives live deployments. A pull request from `develop` into `production` is opened at the end of each sprint after all sprint tests pass.

2.7.3 Automated Deployment Pipeline

Vercel is connected directly to the GitHub repository. Every merge into the `production` branch triggers an automatic Vercel build and deployment, as described in the following steps:

1. Developer pushes feature work to a feature branch and opens a pull request into `develop`.
2. After review, the feature branch is merged into `develop`.
3. At sprint end, `develop` is merged into `production` via a pull request.
4. Vercel detects the push to `production`, runs the Turborepo build pipeline, and publishes the updated application to `collectra.xyz` automatically.

5. Vercel also generates a unique preview Uniform Resource Locator for every pull request, enabling stakeholder review before merging to production.

This pipeline ensures that the live environment always reflects a tested, reviewed state, and that every deployment is traceable to a specific Git commit.

2.8 End-to-End User Journeys

Beyond isolated screens, Collectra supports three repeatable journeys that structure the internship demonstrations and acceptance tests.

2.8.1 Manager Journey

A manager authenticates, creates or selects a workspace, invites agents, and configures campaigns. After agents import debts, the manager monitors progress from the dashboard, filters by campaign, and reviews action history for audit purposes. When payments occur, the manager verifies invoice availability and email tracking indicators. This journey validates workspace isolation, role-based access control, and reporting value.

2.8.2 Agent Journey

An agent authenticates, opens a workspace campaign, and works from the campaign debt table. Imports and updates use a transactional CSV importer and enqueue post-commit background jobs (for example, requesting Brevo to send emails). Brevo is the provider that sends those emails and any post-payment receipts — agents do not send emails or generate receipts directly. During calls agents record promises, update debt statuses, and may initiate a Stripe checkout (the system sets a ‘pendingStripeSessionId‘ lock); agents cannot perform public customer actions. Every state-changing action is logged to ‘CustomerActionHistory‘. The interface keeps common tasks reachable in two or three clicks.

2.8.3 Customer Journey

A customer receives a signed link, opens the public debt page without an account, reviews the amount and due information, optionally submits a promise date, and may start Stripe Checkout. After payment, the customer is redirected to verification and can open the hosted invoice. This journey validates public Application Programming Interface endpoints, token expiration, and idempotent payment confirmation.

2.8.4 Journey Validation Matrix

Table 2.8 maps each journey to the sprint that introduced its core capabilities and the primary evidence used during acceptance reviews (screenshots, Postman collections, and live demos on `collectra.xyz`).

Journey	Sprint	Validation Evidence
Manager onboarding and supervision	1, 3, 4	Workspace invitation flow, dashboard indicators, invoice and tracking overview
Agent daily collection work	2, 3	Comma-Separated Values import, status filters, promise dialog, personal link generation
Customer self-service payment	2, 4	Public debt page, Stripe checkout, verification polling, hosted invoice

Table 2.8: User Journey Validation Matrix

These journeys were rehearsed before each sprint review so stakeholders could follow a coherent story rather than isolated features. For the defense, the same order (manager → agent → customer) provides a logical narrative that mirrors real collection operations.

Conclusion

This chapter defined the project actors and requirements, described the work environment and technology stack, introduced the Scrum methodology and project timeline, presented the general architecture, and detailed the Vercel-based deployment pipeline. The following chapters present each sprint in detail.

3 Sprint 1: Core Authentication and Workspace Setup

Introduction

Sprint 1 establishes the secure foundation of the Collectra platform. It covers user authentication, workspace creation, and the member invitation flow. These features are prerequisites for all subsequent sprints, as every resource in the system is scoped to a workspace and every action requires an authenticated session.

3.1 Sprint 1 Backlog

Table 3.1 lists the user stories selected for Sprint 1.

ID	User Story	Period	Priority
S1-01	As a user I want to register with my email and password so that I can access the platform.	Weeks 3–4	High
S1-02	As a user I want to log in securely so that my session is protected by an Hypertext Transfer Protocol-only cookie.	Weeks 3–4	High
S1-03	As a manager or agent I want to create a workspace so that my team data is isolated.	Weeks 3–4	High
S1-04	As a manager I want to invite team members by email so that agents can join my workspace.	Weeks 3–4	High
S1-05	As an invited user I want to accept an invitation so that I can join the workspace.	Weeks 3–4	High
S1-06	As a user I want to log out securely so that my session is properly closed.	Weeks 3–4	Medium
S1-07	As a user I want to delete my account so that all my workspace and associated data are permanently removed.	Weeks 3–4	High

Table 3.1: Sprint 1 Backlog

3.2 Functional Specification

3.2.1 Use Case Diagram

Figure 3.1 shows the use case diagram for Sprint 1, covering registration, login, workspace creation, and the invitation flow.

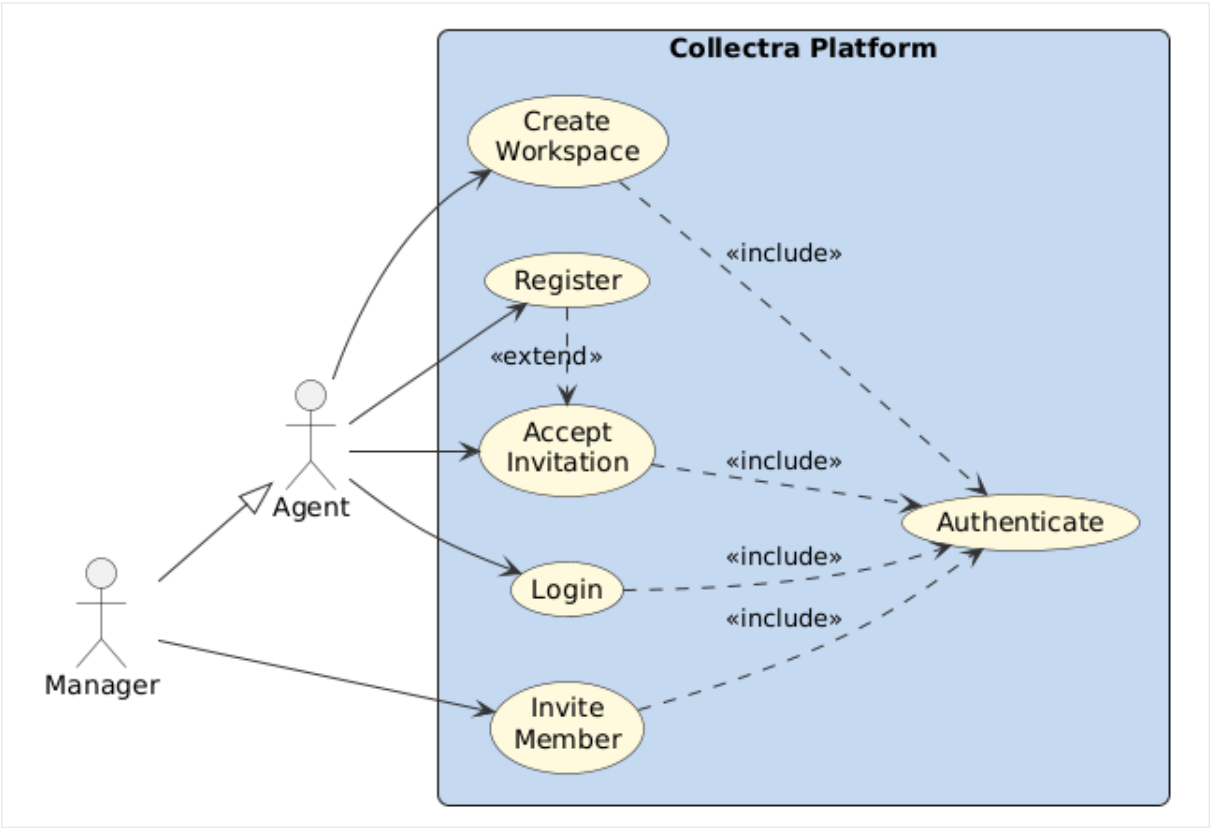

Figure 3.1: Use Case Diagram – Sprint 1

Use Case	User Registration
Actor	Manager, Agent
Precondition	User has a valid email address and is not already registered.
Main Flow	1. User opens the registration page. 2. User enters name, email, and password. 3. System validates inputs via Zod schema. 4. System creates account via Supabase Auth. 5. User is redirected to workspace creation.
Postcondition	User account is created; session is active.
Alternative	If the email already exists, system displays an error and prompts login.

Table 3.2: Use Case Description – User Registration

3.2.2 Textual Description of Use Cases

- User Registration

- Workspace Creation

Use Case	Workspace Creation
Actor	Manager, Agent
Precondition	User is authenticated and does not yet belong to a workspace.
Main Flow	1. Manager navigates to workspace setup. 2. Manager enters workspace name and optional settings. 3. System creates the workspace and assigns the manager as Owner. 4. Manager is redirected to the workspace dashboard.
Postcondition	Workspace is created; manager holds the Owner role.
Alternative	If the workspace name is already taken, system displays an error.

Table 3.3: Use Case Description – Workspace Creation

- Member Invitation

Use Case	Member Invitation
Actor	Manager
Precondition	Manager is authenticated and owns or manages a workspace.
Main Flow	1. Manager opens the team management screen. 2. Manager enters the invitee's email and selects a role (Agent or Manager). 3. System generates a unique invitation token. 4. System sends an invitation email with a signed acceptance link. 5. Invitation appears in the list with status *Pending*.
Postcondition	Invitation record is saved; invitee receives an email.
Alternative	If the email is invalid or the user is already a member, an error is shown.

Table 3.4: Use Case Description – Member Invitation

3.3 Design

3.3.1 Class Diagram

Figure 3.2 shows the class diagram for Sprint 1, covering the `User`, `Workspace`, `WorkspaceMember`, and `Invitation` entities.

3.3.2 Sequence Diagrams

Figures 3.3–3.5 illustrate the authentication, invitation, and acceptance flows respectively.

3.4 Implementation

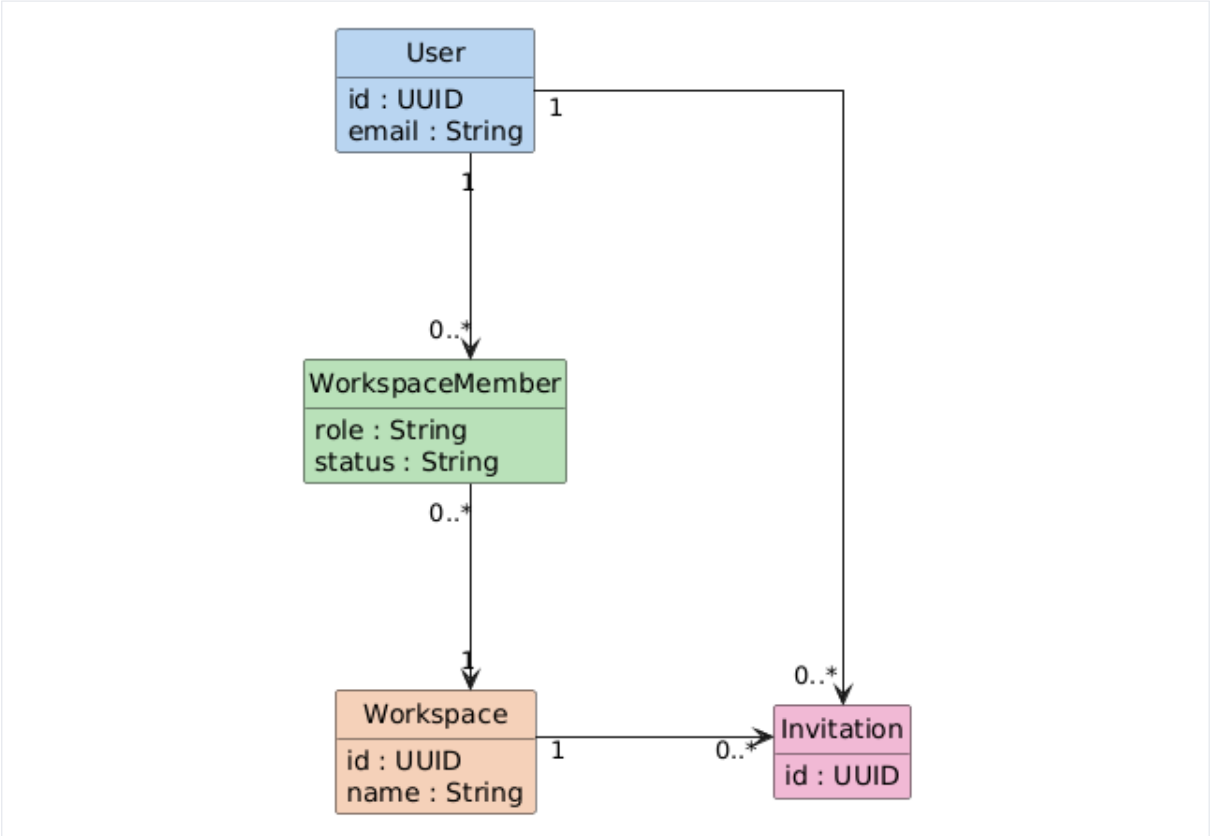

Figure 3.2: Class Diagram – Sprint 1

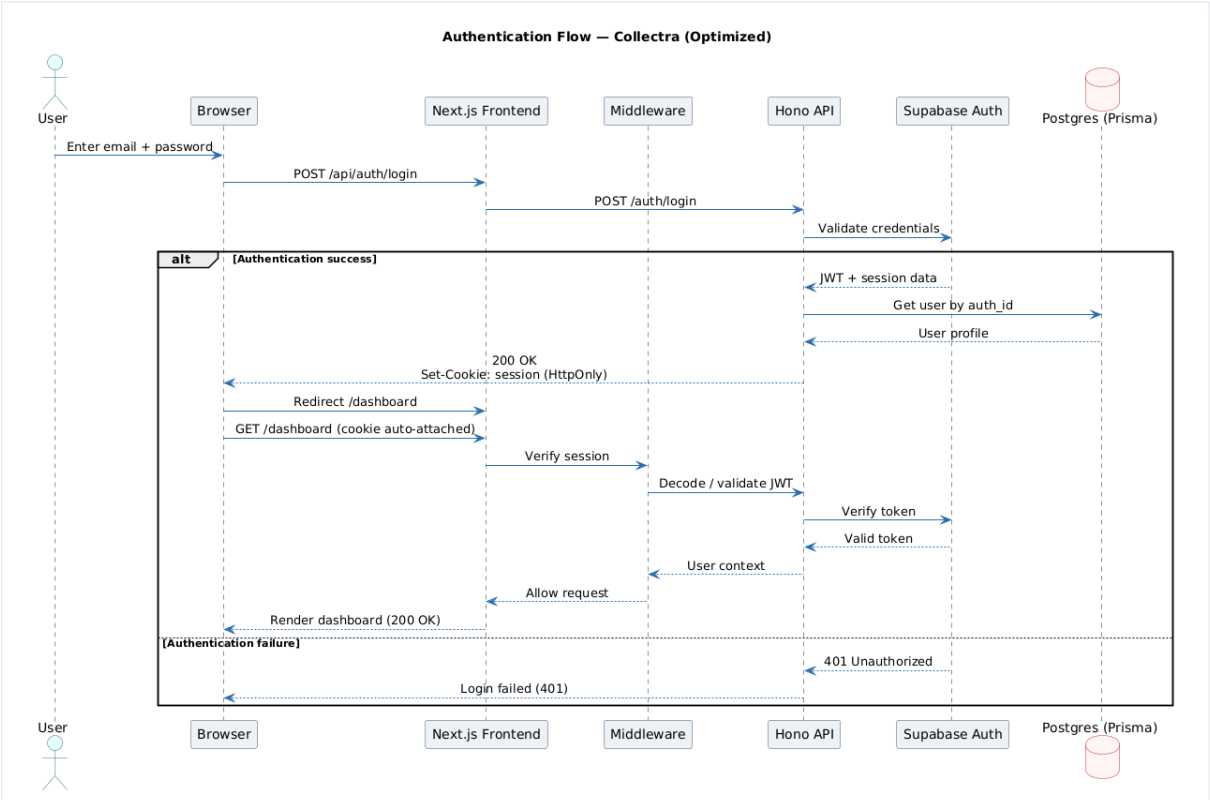

Figure 3.3: Sequence Diagram – Authentication Flow

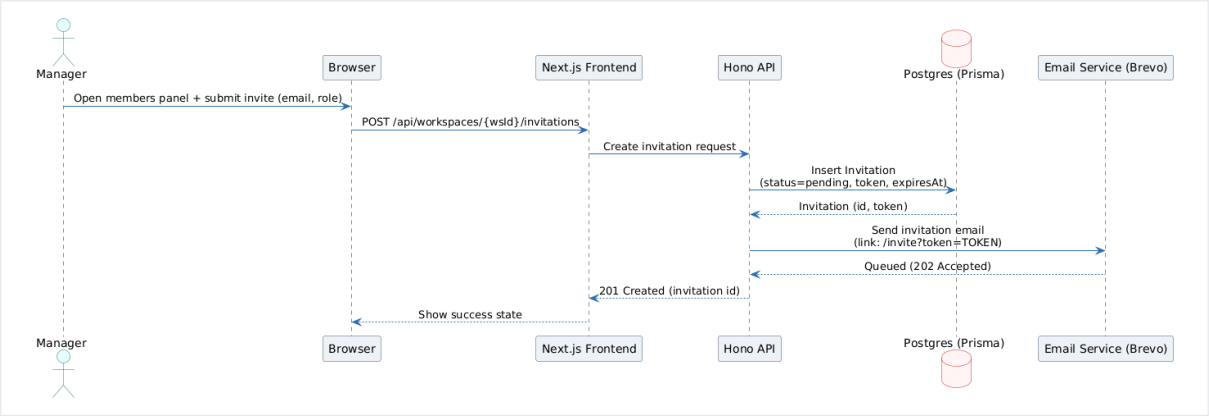

Figure 3.4: Sequence Diagram – Invite Member Flow

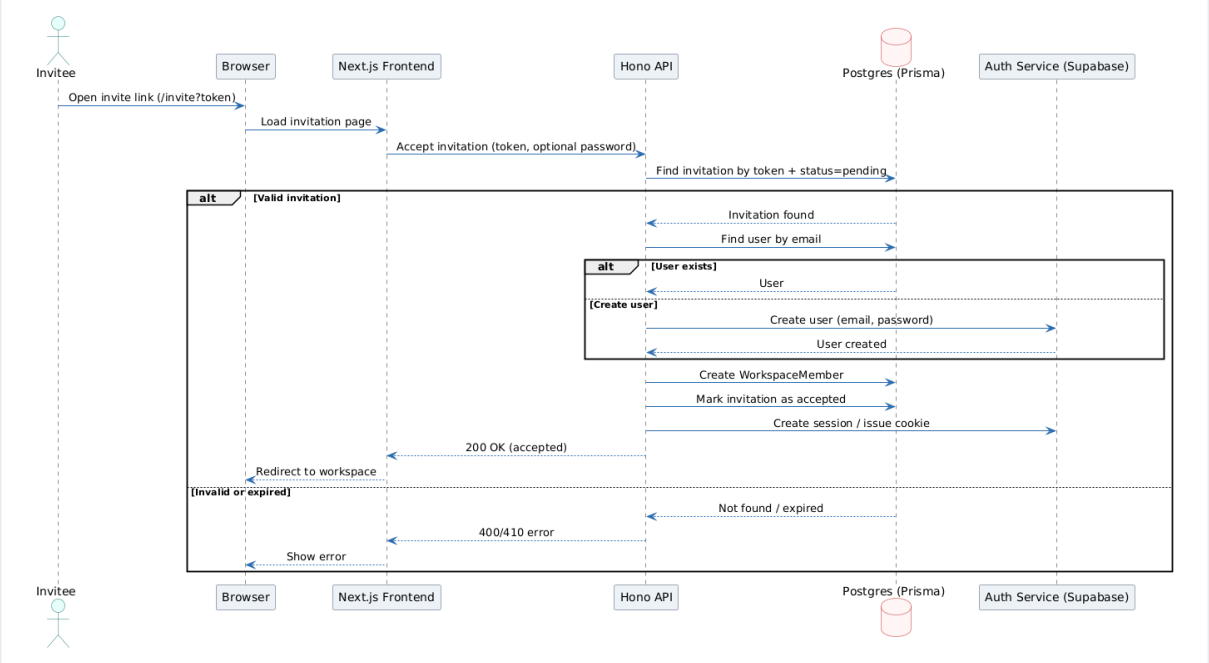

Figure 3.5: Sequence Diagram – Accept Invitation Flow

3.4.1 Sprint 1 Interface

The Sprint 1 interface covers three main screens: authentication, workspace creation, and team management. Figures 3.6–3.8 show these screens.

3.4.2 Sprint 1 Business Logic

Authentication

Authentication is handled by Supabase Auth using email and password. On a successful login, the backend sets an Hypertext Transfer Protocol-only session cookie containing the user's JSON Web Token. A Next.js middleware layer checks this cookie on every protected route and page, redirecting unauthenticated requests to the login screen. This approach prevents

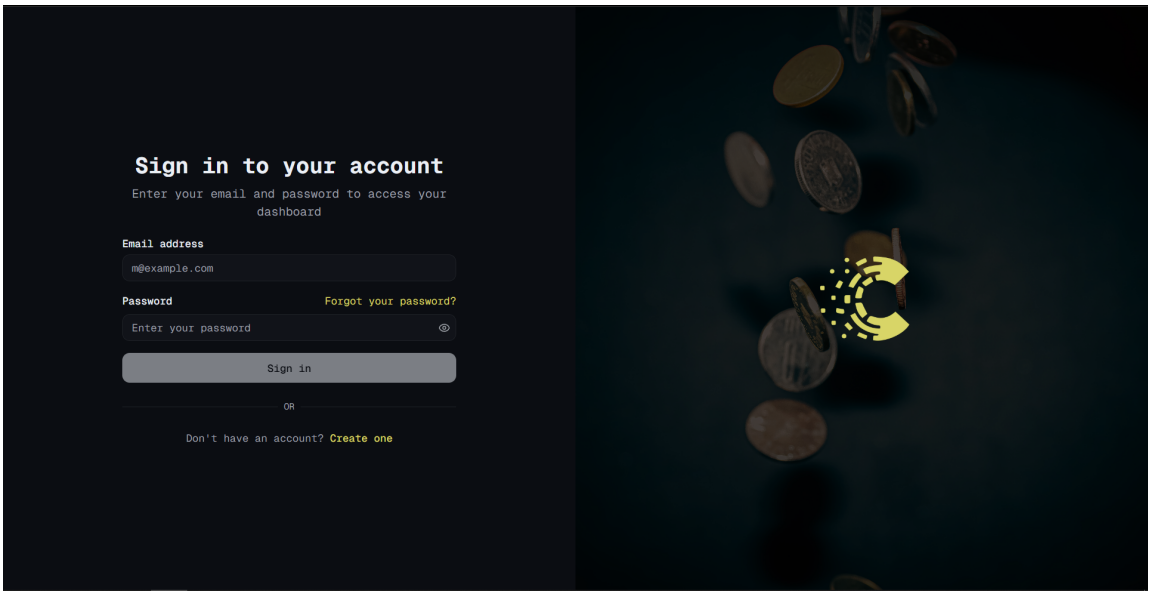

Figure 3.6: Login Screen

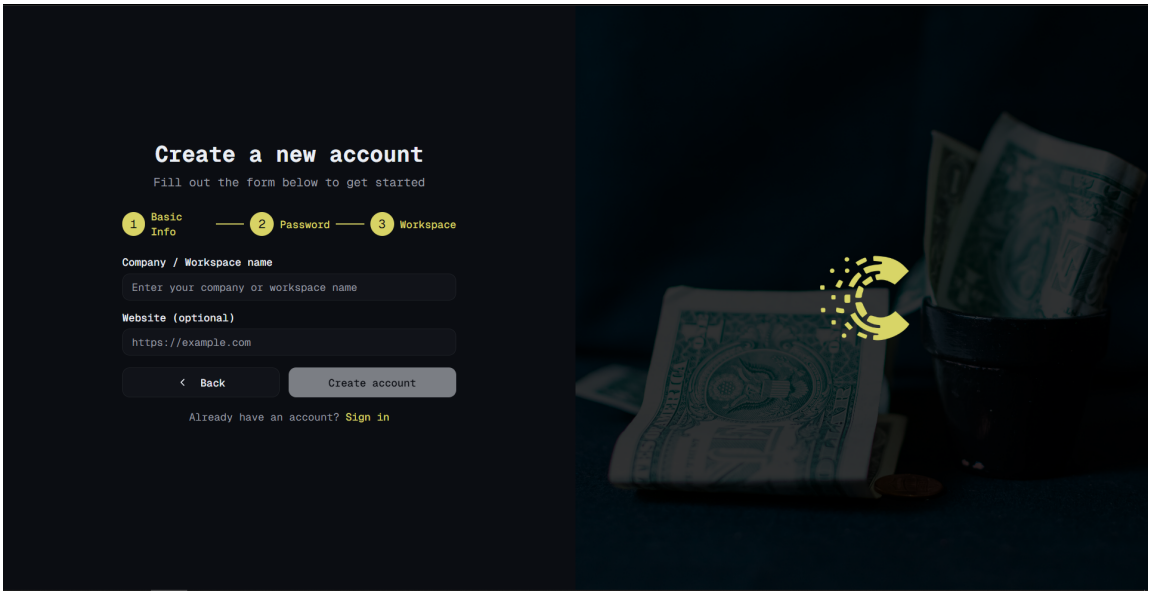

Figure 3.7: Workspace Creation Screen

token theft via JavaScript and protects all workspace data behind a single, centrally managed guard.

Workspace Isolation

Every Application Programming Interface endpoint that accesses workspace resources validates a `workspaceId` guard before processing any data. The system verifies that the authenticated user is an active member of the requested workspace, ensuring that data from one workspace is never visible to users of another.

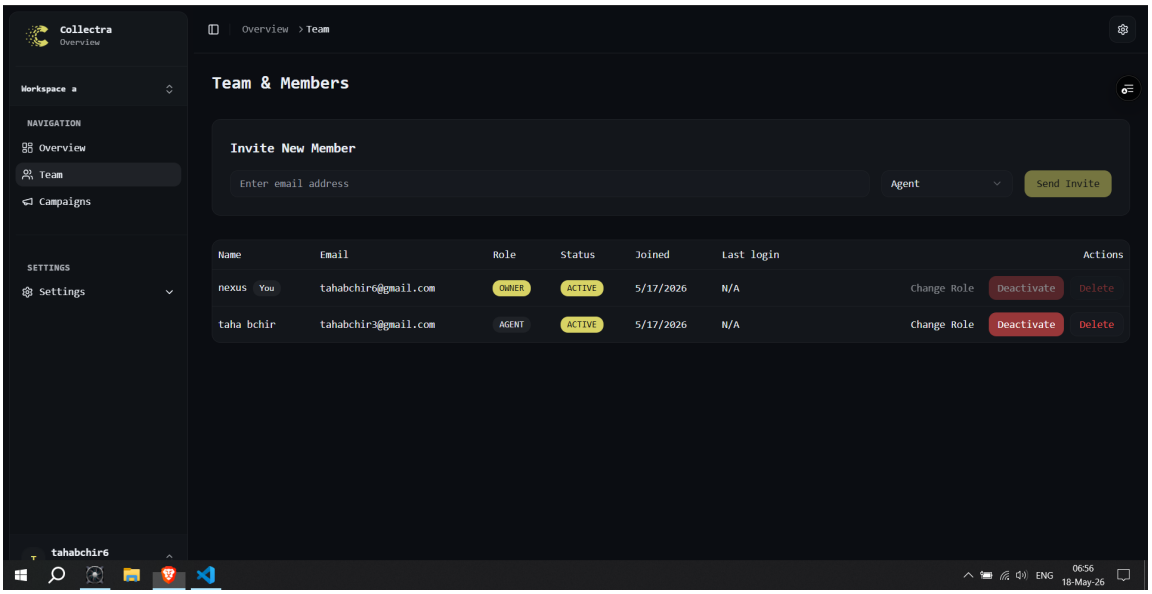

Figure 3.8: Team Management Screen

Invitation Flow

When a manager invites a user, the system:

1. generates a unique, time-limited invitation token;
2. stores the invitation record with status *Pending* in the database;
3. sends an email containing a signed acceptance link;
4. on acceptance, creates the workspace membership with the assigned role and marks the invitation as *Accepted*.

3.4.3 Sprint 1 – Test Results

Table 3.5 presents the functional tests executed at the end of Sprint 1.

3.4.4 Sprint 1 Retrospective

The main success of Sprint 1 was establishing workspace isolation early, which simplified every later feature. The team improved estimation for invitation flows after discovering email template edge cases. For Sprint 2, we prepared standardized Comma-Separated Values samples and clarified status transition rules in the backlog before development started.

From a process perspective, Sprint 1 validated the monorepo toolchain: Turborepo caching reduced local build times, and preview deployments allowed CyberBridge reviewers to test invitations without database access. Technical debt was intentionally limited to polish items (password reset messaging, clearer error copy) so Sprint 2 could focus on domain complexity. The retrospective action items recorded in Linear were: (i) document middleware contract for

ID	Test Description	Expected Result	Actual
T1-01	Register with valid email and password	Account created, redirect to workspace setup	As expected
T1-02	Register with an already-used email	Error message displayed, no duplicate account	As expected
T1-03	Login with correct credentials	Hypertext Transfer Protocol-only cookie set, redirect to dashboard	As expected
T1-04	Login with wrong password	Error message, no cookie set	As expected
T1-05	Access protected page without session	Redirect to login screen	As expected
T1-06	Create workspace with a valid name	Workspace created, Owner role assigned	As expected
T1-07	Invite a user by email with role Agent	Invitation email sent, status set to Pending	As expected
T1-08	Accept invitation via signed link	Membership created, redirect to workspace	As expected
T1-09	Log out from an active session	Cookie cleared, redirect to login	As expected
T1-10	Access another workspace without membership	403 Forbidden returned, no data exposed	As expected

Table 3.5: Sprint 1 – Functional Test Results

workspace scoping, (ii) add Postman tests for forbidden cross-workspace access, and (iii) capture screenshots for the report appendix while flows were still fresh.

Conclusion

Sprint 1 delivered a fully tested authentication system, workspace creation, and member invitation workflow. The decision to enforce workspace isolation at the middleware level – rather than in individual route handlers – proved to be the key architectural choice that simplified all subsequent data access patterns. The next chapter presents Sprint 2, which extends the platform with campaign and debt management.

4 Sprint 2: Campaign and Debt Management

Introduction

Sprint 2 builds on the authenticated workspace foundation from Sprint 1. It introduces campaign management, bulk Comma-Separated Values debt import, debt status tracking across a defined lifecycle, and the generation of JSON Web Token-secured customer personal links. The most technically demanding aspect of this sprint was designing a status transition system that is strict enough to prevent invalid lifecycle jumps while remaining easy to extend in future sprints.

4.1 Sprint 2 Backlog

Table 4.1 and Table 4.2 list the user stories selected for Sprint 2.

ID	User Story	Period	Priority
S2-01	As a manager I want to create campaigns so that related debts are grouped logically.	Weeks 5–6	High
S2-02	As an agent I want to import debts from a Comma-Separated Values file so I can add many records at once.	Weeks 5–6	High
S2-03	As an agent I want the importer to handle common Comma-Separated Values variations and large files so imports are reliable and fast.	Weeks 5–6	High
S2-04	As an agent I want to view and filter debts by status and campaign so I can prioritise work.	Weeks 5–6	High
S2-05	As an agent I want the system to enforce correct status changes so debt lifecycles stay consistent.	Weeks 5–6	High
S2-06	As an agent I want to generate secure, time-limited customer links so customers can view and act on their debt without an account.	Weeks 5–6	High

Table 4.1: Sprint 2 Backlog (part 1)

ID	User Story	Period	Priority
S2-07	As an agent I want notification emails sent after import so customers receive outreach automatically.	Weeks 5–6	Medium
S2-08	As a customer I want to open a secure link to view my debt and make a promise or payment without registering.	Weeks 5–6	Medium

Table 4.2: Sprint 2 Backlog (part 2)

4.2 Functional Specification

4.2.1 Use Case Diagram

Figure 4.1 shows the use case diagram for Sprint 2.

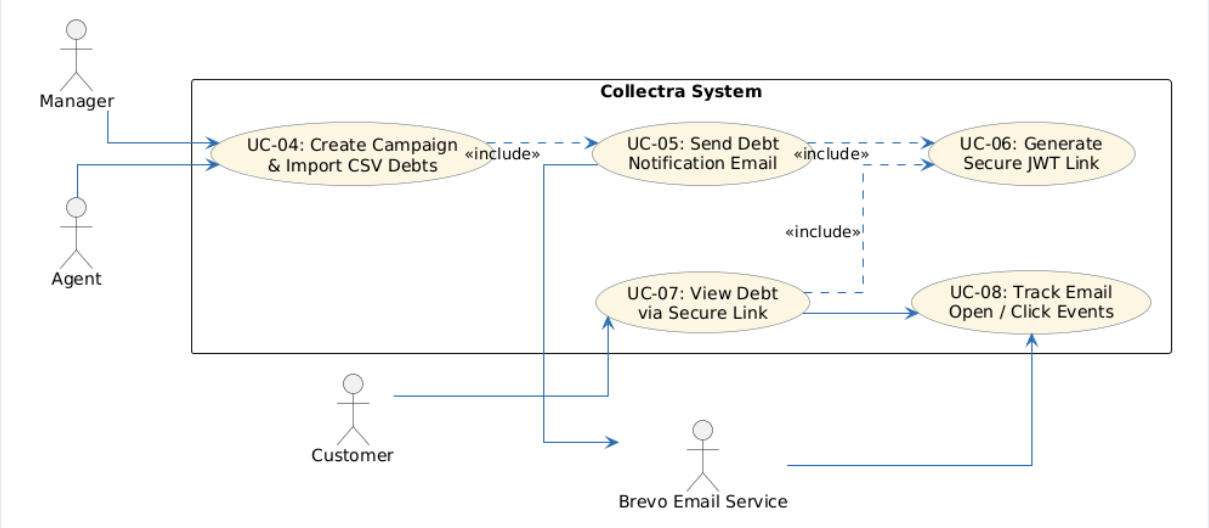

Figure 4.1: Use Case Diagram – Sprint 2

4.2.2 Textual Description of Use Cases

Campaign Creation

Use Case	Campaign Creation
Actor	Manager, Agent
Precondition	User is authenticated and is a member of the workspace.
Main Flow	1. User navigates to the Campaigns section. 2. User clicks Create Campaign and fills in name and description. 3. System validates and saves the campaign record. 4. Campaign appears in the list with status *Active*.
Postcondition	Campaign is created and ready to receive debt records.
Alternative	If required fields are missing, system highlights the validation errors.

Table 4.3: Use Case Description – Campaign Creation

Comma-Separated Values Debt Import

Customer Personal Link Generation

4.3 Design

4.3.1 Class Diagram

Figure 4.2 shows the class diagram for Sprint 2, adding the `Campaign`, `Debt`, and `CustomerLink` entities.

Use Case	Comma-Separated Values Debt Import
Actor	Agent
Precondition	A campaign exists; user has Agent or Manager role.
Main Flow	1. Agent opens the campaign and selects Import Comma-Separated Values. 2. Agent uploads a Comma-Separated Values file with debt records. 3. System parses, validates, and batch-inserts records into the database. 4. A summary of imported, skipped, and error rows is displayed.
Postcondition	Valid debt records are created with status *Imported*.
Alternative	Rows with missing or invalid fields are skipped and counted in the error report.

Table 4.4: Use Case Description – Comma-Separated Values Debt Import

Use Case	Customer Personal Link Generation
Actor	Agent
Precondition	A debt record exists; user has Agent or Manager role.
Main Flow	1. Agent opens a debt record and clicks Generate Link. 2. System creates a signed JSON Web Token containing the debt ID and a 48-hour expiration timestamp. 3. System stores the link record and returns the personal Uniform Resource Locator. 4. Agent shares the link with the customer (email or phone).
Postcondition	A 48-hour personal link is generated and stored.
Alternative	If an active link already exists for that debt, the system returns it instead of creating a new one.

Table 4.5: Use Case Description – Customer Personal Link Generation

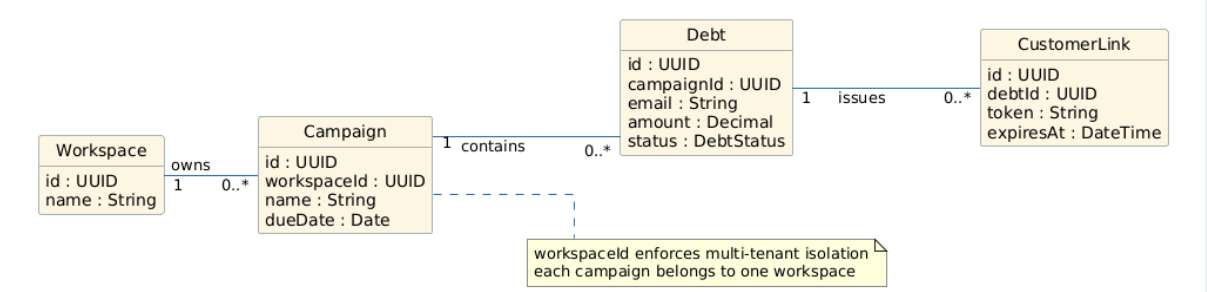

Figure 4.2: Class Diagram – Sprint 2

4.3.2 Sequence Diagrams

Figure 4.3 shows the campaign and Comma-Separated Values import flow. Figure 4.4 shows the customer personal link flow.

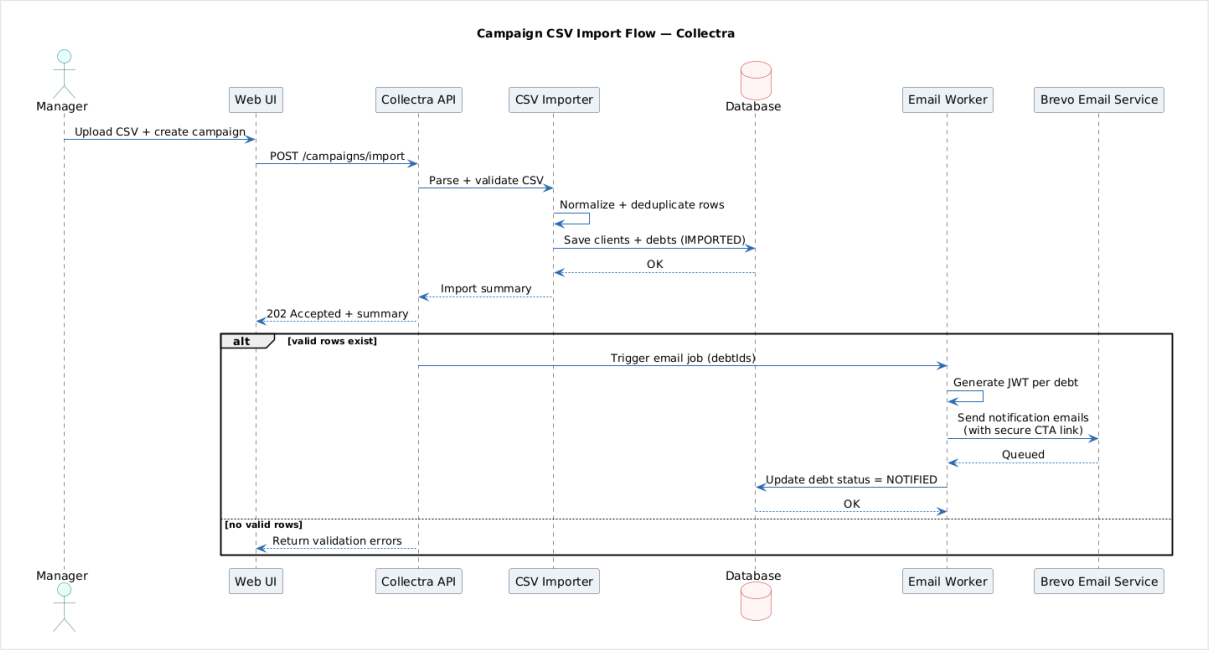

Figure 4.3: Sequence Diagram – Campaign and Comma-Separated Values Import Flow

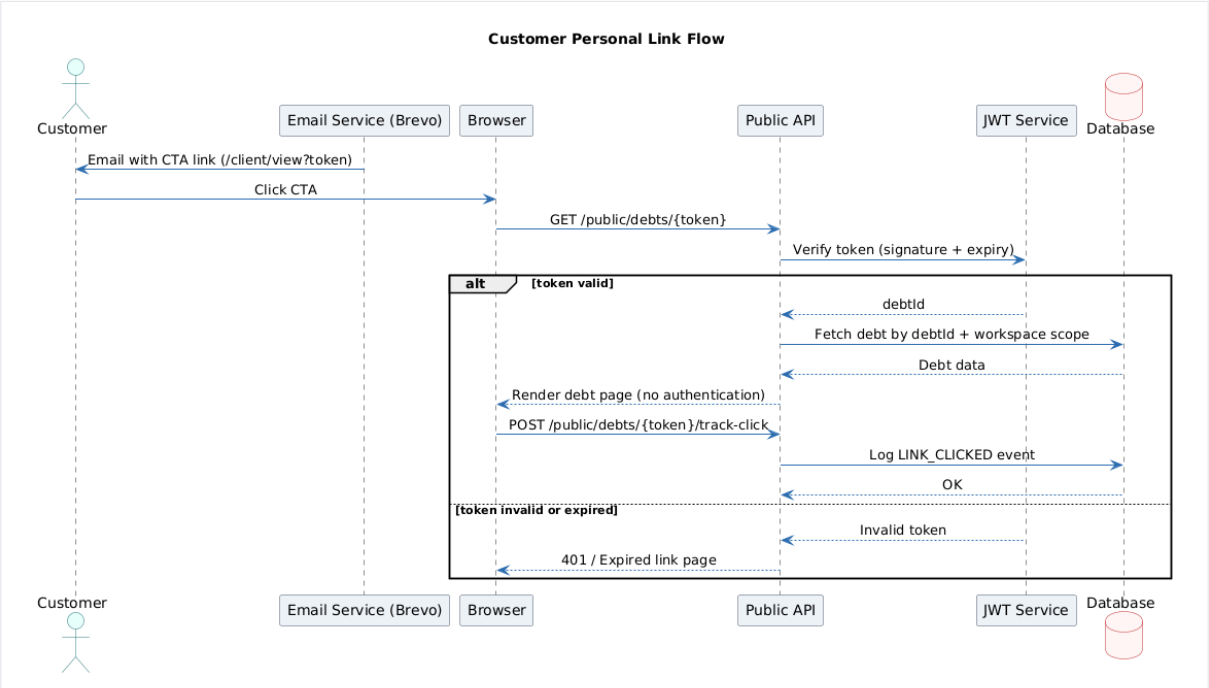

Figure 4.4: Sequence Diagram – Customer Personal Link Flow

4.4 Implementation

4.4.1 Sprint 2 Interface

The Sprint 2 interface covers the campaign list, the debt table with status filters, the Comma-Separated Values import dialog, and the customer personal link screen.

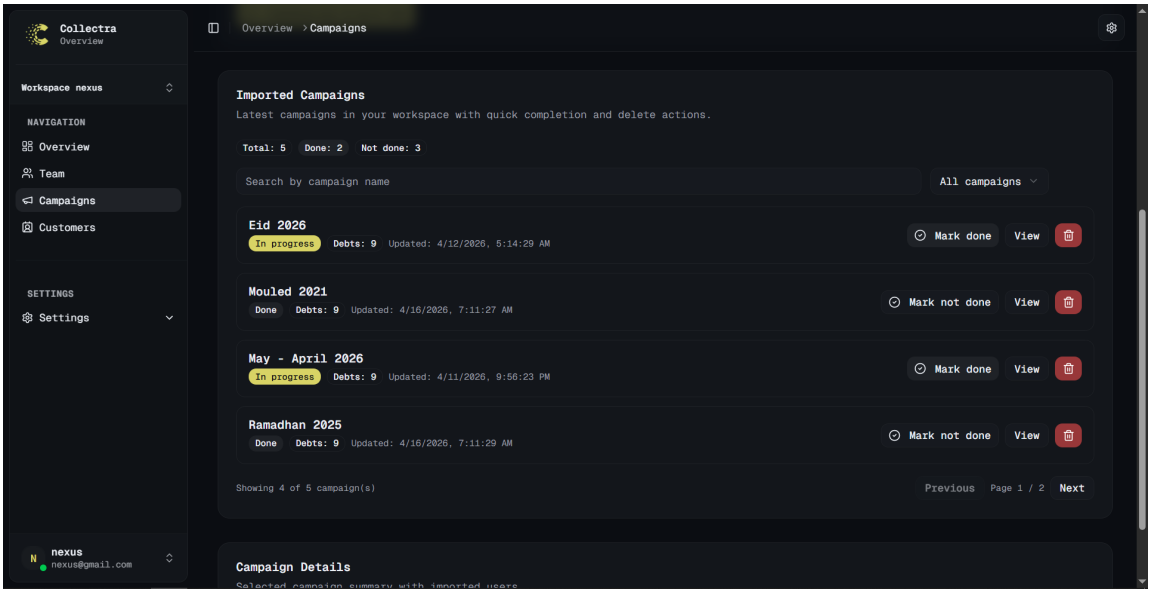

Figure 4.5: Campaign List – Sprint 2

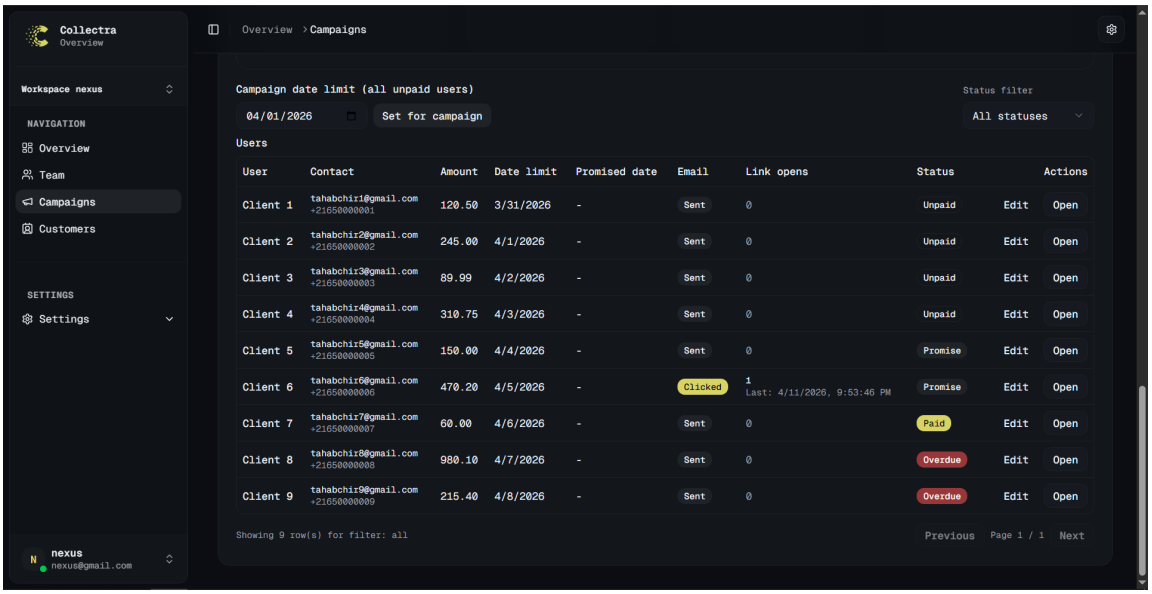

Figure 4.6: Debt Table with Status Filters – Sprint 2

4.4.2 Sprint 2 Business Logic

Debt Lifecycle

The debt lifecycle follows a strict sequence of status transitions:

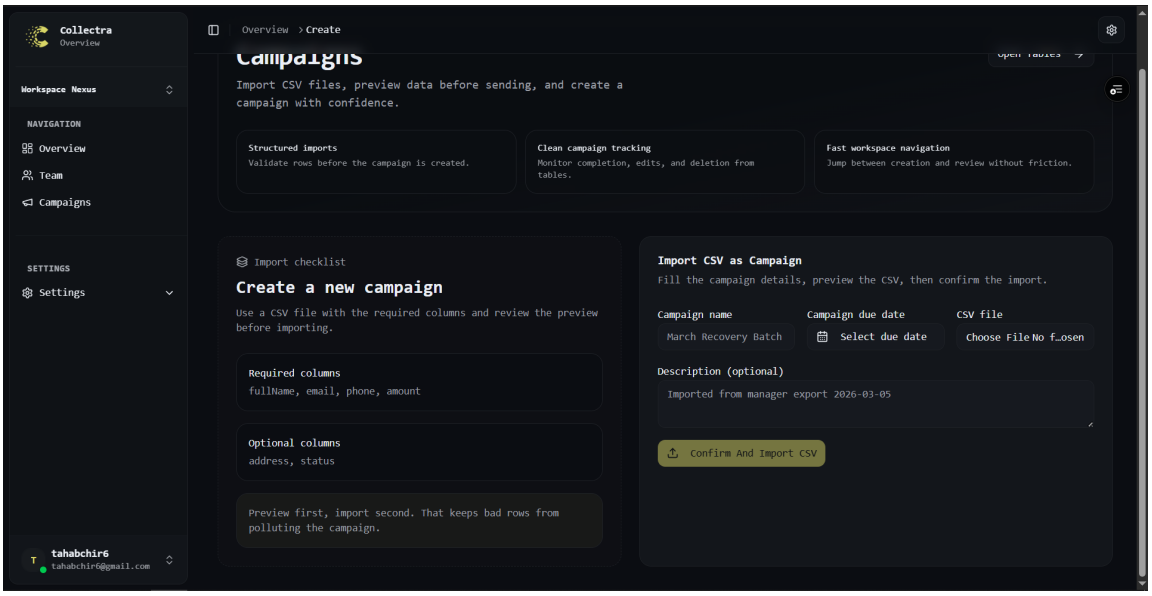

Figure 4.7: Comma-Separated Values Import Dialog – Sprint 2

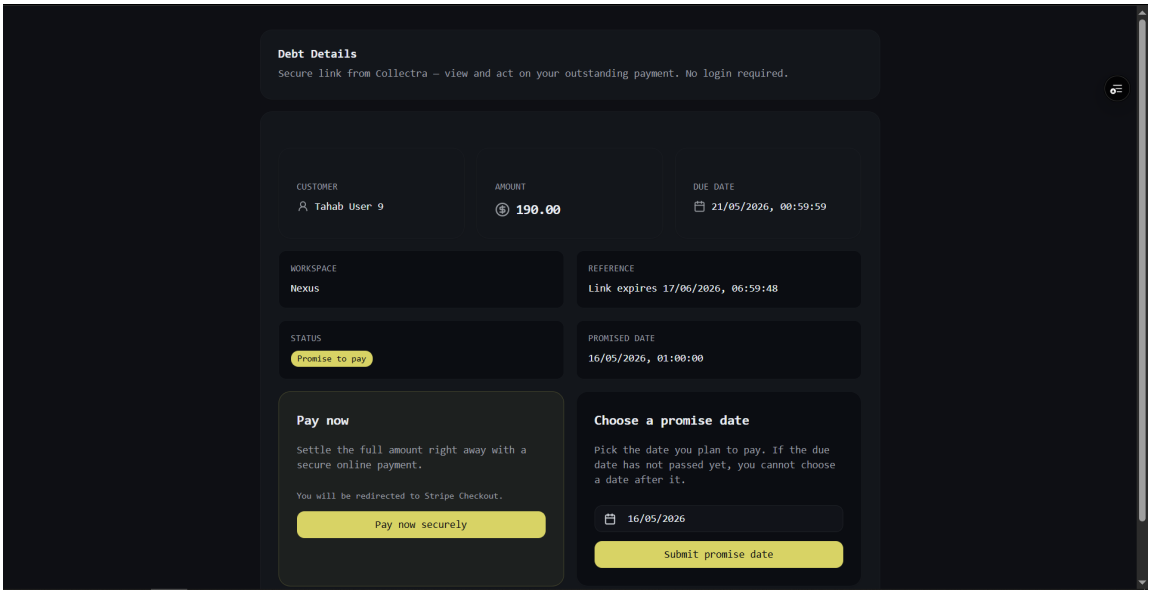

Figure 4.8: Customer Personal Link Screen – Sprint 2

`Imported` → `Notified` → `Promised` → `Paid`

A debt may also move to `OVERDUE_AFTER_PROMISE` if the recorded due date or a promise date passes without a payment being confirmed. The backend enforces allowed transitions and prevents invalid jumps; this logic is implemented in the Debts and Overdue services (`apps/api/src/services/debts.ts`, `apps/api/src/services/overdue-debts.ts`).

Comma-Separated Values Import Logic

The Comma-Separated Values import endpoint implements a tolerant, production-ready importer:

1. receives the file as a multipart/form-data upload;
2. detects the delimiter (comma, semicolon, or tab) and parses quoted values robustly;
3. resolves header aliases (for example `fullname`, `name`, `clientname` → full name; `montant`/`amount` → amount) and validates required columns (full name, email, phone, amount);
4. normalizes fields (email lowercasing, amount parsing) and maps textual status values using a canonical mapping (see `STATUS_MAPPING` in `apps/api/src/services/campaigns.ts`);
5. matches existing clients by case-insensitive email; for unseen emails the importer prepares a batch of pending clients and inserts them using `createMany` in chunks (safe chunk size used in code: 500);
6. inserts debts using `createMany` in chunked batches inside a transaction to avoid timeouts;
7. returns a detailed import summary (total rows, imported rows, skipped rows with reasons) and a list of per-debt notifications to attempt via Brevo.

JSON Web Token Customer Links

Personal links are implemented as signed JSON Web Tokens (Hash-based Message Authentication Code SHA-256). The token encodes the debt id in the `sub` claim and is signed using the server secret. The implementation issues tokens with a default expiry of 30 days (see `EXPIRY_DAYS` in `apps/api/src/lib/customer-jwt.ts`). Token verification is stateless (no DB row required) and performed by `verifyCustomerToken` when the public link is accessed.

Customer actions performed via the public link (link click, link open, promise creation, payment initiation) are recorded in `customerActionHistory` and Brevo event logs where applicable.

4.4.3 Sprint 2 – Test Results

Table 4.6 presents the functional tests executed at the end of Sprint 2.

4.4.4 Sprint 2 Retrospective

Import performance and status-transition rules were the highest-risk topics. Chunked inserts and explicit transition guards proved sufficient for production-like files. The retrospective highlighted the need for better import error reporting in the user interface, which guided Sprint 3 dashboard and history priorities.

Sprint 2 also confirmed that customer personal links must remain independent from internal workspace routing: public pages use a dedicated layout without navigation chrome, which reduces accidental exposure of manager features. Performance tests on preview environments showed that imports above three thousand rows remain acceptable when batched, but the user interface now surfaces progress indicators to manage agent expectations during long uploads.

ID	Test Description	Expected Result	Actual
T2-01	Create campaign with valid name and description	Campaign saved, status Active	As expected
T2-02	Create campaign with missing required fields	Validation error displayed	As expected
T2-03	Import valid Comma-Separated Values with 50 debt records	50 records created, status Imported	As expected
T2-04	Import Comma-Separated Values with 2 invalid rows out of 10	8 records created, 2 counted as errors	As expected
T2-05	Update debt status from Imported to Notified	Status updated, action logged	As expected
T2-06	Attempt invalid status transition (Imported to Paid)	400 Bad Request returned	As expected
T2-07	Generate customer personal link for a debt	Signed Uniform Resource Locator returned, 30-day expiry set	As expected
T2-08	Open valid personal link as customer	Debt details displayed, no login required	As expected
T2-09	Open expired personal link	401 Unauthorized returned	As expected
T2-10	Generate link when one already exists	Existing active link returned	As expected

Table 4.6: Sprint 2 – Functional Test Results

Action items for Sprint 3 included richer per-row error export, campaign-level filters on the debt table, and history entries whenever status changes occur in bulk.

Conclusion

Sprint 2 delivered a fully tested campaign and debt management module, including bulk Comma-Separated Values import, a strictly enforced debt status lifecycle, and 30-day JSON Web Token-secured customer personal links. The key design decision – validating all status transitions server-side rather than relying on frontend state – ensured data integrity throughout the lifecycle. The next chapter presents Sprint 3, which focuses on collaboration and reporting.

5 Sprint 3: Audit Trail, Payment Promises, and Manager Dashboard

Introduction

Sprint 3 focuses on giving managers full operational visibility and control over the platform. It adds action history tracking for complete auditability, payment promise recording to extend the debt lifecycle, the manager dashboard for real-time collection monitoring, and role-based permission enforcement to protect sensitive operations. The central design challenge was aggregating debt data across campaigns efficiently without introducing N+1 query patterns on the dashboard endpoint.

5.1 Sprint 3 Backlog

Table 5.1 lists the user stories selected for Sprint 3.

ID	User Story	Period	Priority
S3-01	As an agent I want to record a payment promise so that the commitment date is tracked.	Weeks 7–8	Medium
S3-02	As a manager I want to view a full action history so that all operations are traceable.	Weeks 7–8	Medium
S3-03	As a manager I want dashboard indicators so that I can monitor overall collection progress.	Weeks 7–8	Medium
S3-04	As a manager I want to filter the dashboard by campaign so that I can compare performance.	Weeks 7–8	Medium
S3-05	As a manager I want role-based permissions enforced on sensitive actions.	Weeks 7–8	High

Table 5.1: Sprint 3 Backlog

5.2 Functional Specification

5.2.1 Use Case Diagram

Figure 5.1 shows the use case diagram for Sprint 3.

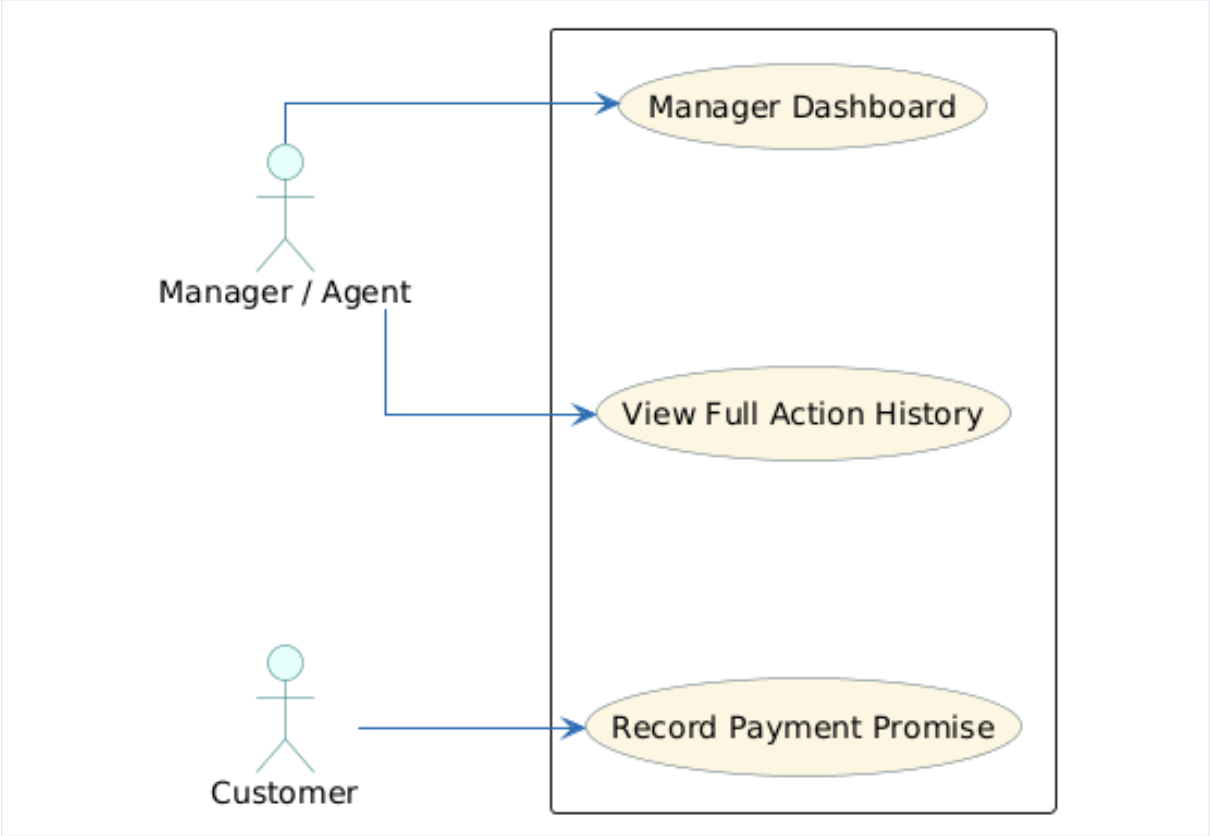

Figure 5.1: Use Case Diagram – Sprint 3

Use Case	UC-07: Record Payment Promise
Actor	Agent
Precondition	Debt exists with status *Notified* or *Imported*.
Main Flow	1. Agent opens the debt record. 2. Agent selects Record Promise and enters the promised payment date. 3. System saves the promise and transitions debt status to *Promised*. 4. Action is written to the history log.
Postcondition	Promise is recorded; debt status is *Promised*.
Alternative	If the promise date is already past, system immediately flags debt as *Overdue*.

Table 5.2: Use Case Description – UC-07: Record Payment Promise

Use Case	UC-08: Manager Dashboard
Actor	Manager
Precondition	Manager is authenticated and the workspace has at least one campaign.
Main Flow	1. Manager opens the Dashboard section. 2. System aggregates debt counts by status per campaign. 3. Dashboard displays key indicators: total, paid, promised, overdue. 4. Manager can filter by campaign or date range.
Postcondition	Manager has a real-time aggregated view of collection progress.
Alternative	If no data exists, dashboard shows an empty state with onboarding guidance.

Table 5.3: Use Case Description – UC-08: Manager Dashboard

5.2.2 Textual Description of Use Cases

UC-07: Record Payment Promise

UC-08: Manager Dashboard

5.3 Design

5.3.1 Class Diagram

Figure 5.2 shows the Sprint 3 class diagram, adding the `PaymentPromise` and `CustomerActionHistory` entities.

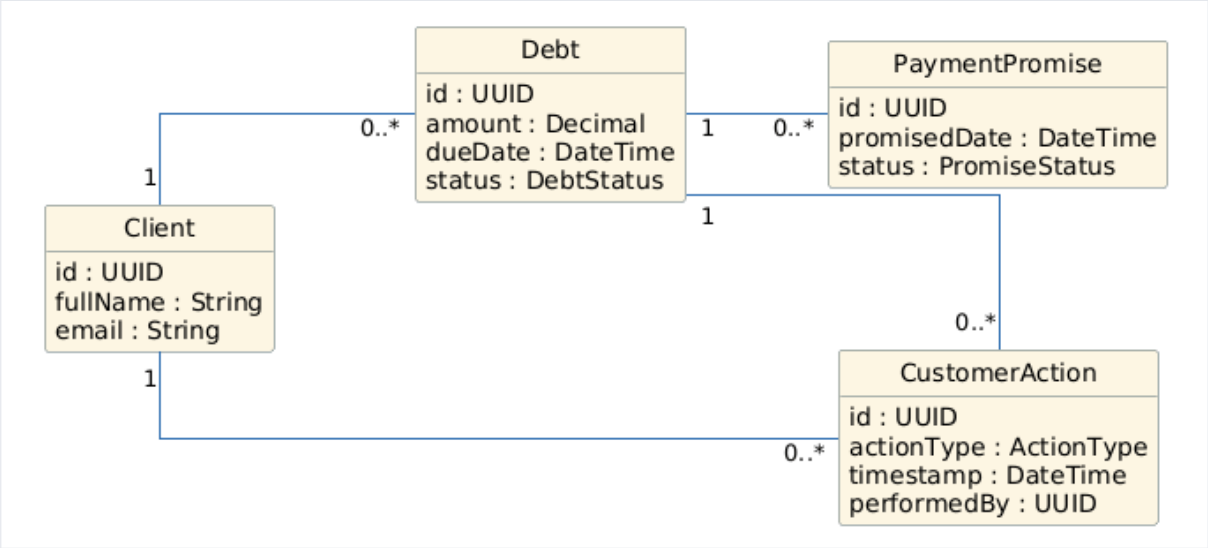

Figure 5.2: Class Diagram – Sprint 3

5.3.2 Sequence Diagrams

Figure 5.3 shows the payment promise recording flow. Figure 5.4 shows the dashboard data aggregation flow.

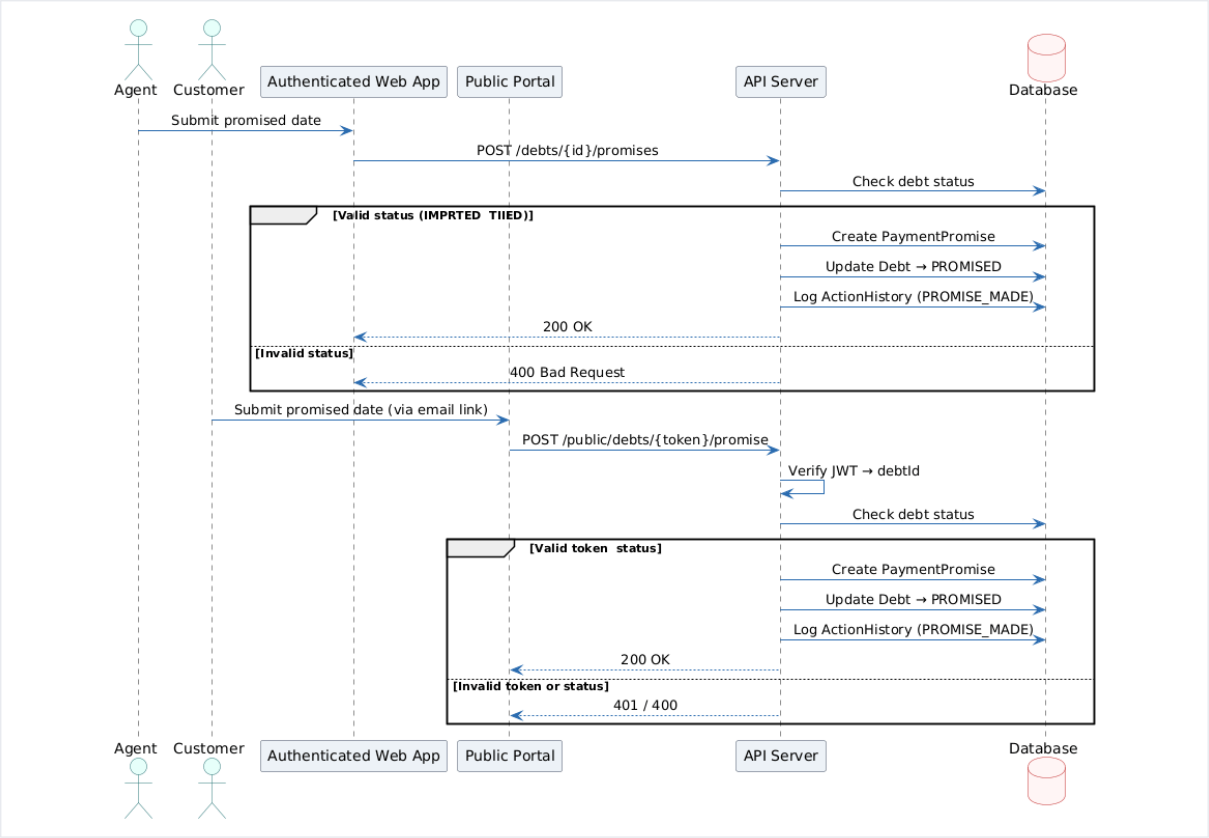

Figure 5.3: Sequence Diagram – Payment Promise Flow

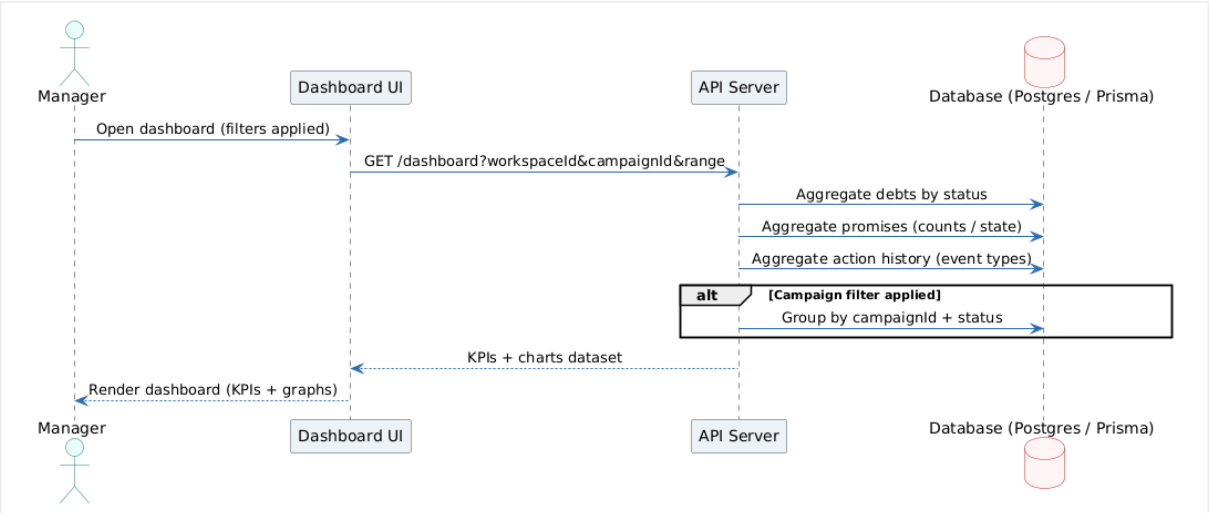

Figure 5.4: Sequence Diagram – Dashboard Data Aggregation

5.4 Implementation

5.4.1 Sprint 3 Interface

Sprint 3 adds three main screens: a payment promise dialog, an action history table, and the manager dashboard with key performance indicators.

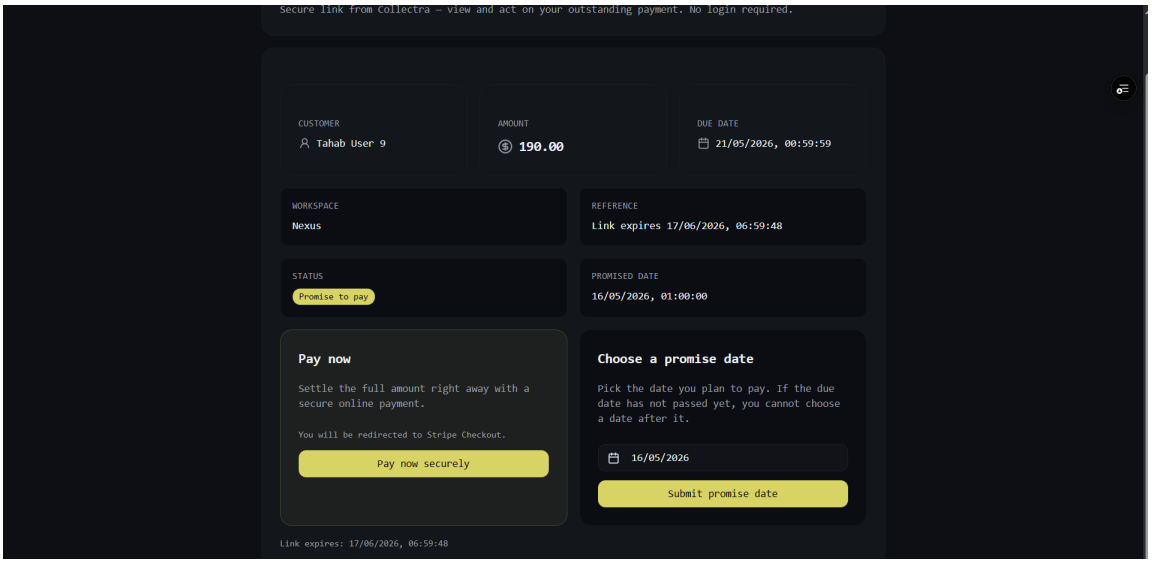

Figure 5.5: Payment Promise Dialog – Sprint 3

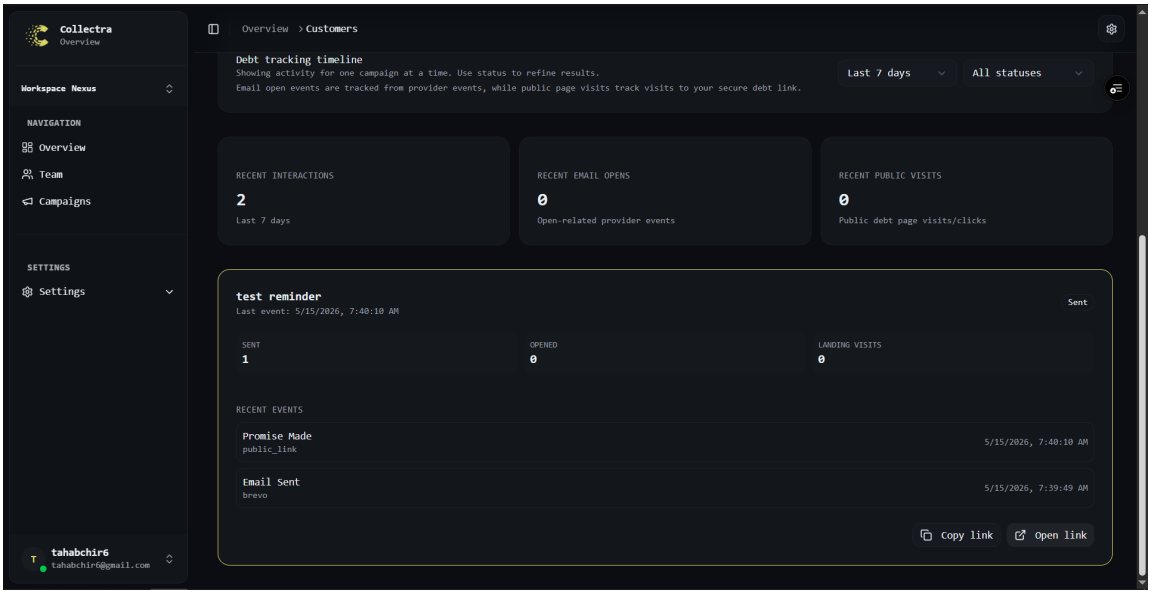

Figure 5.6: Action History Table – Sprint 3

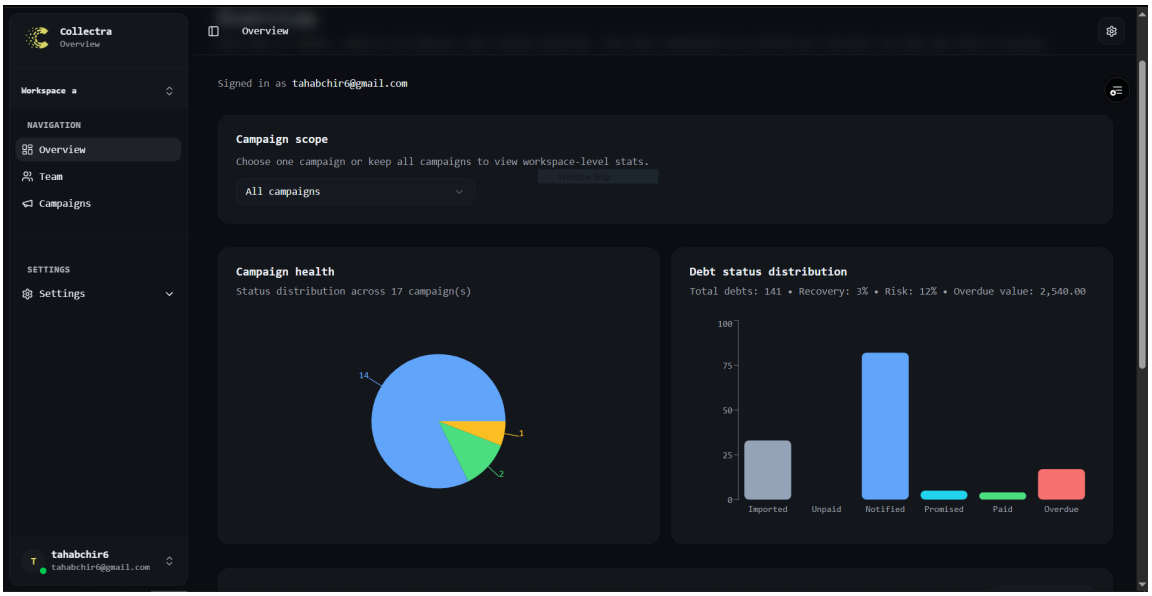

Figure 5.7: Dashboard – Sprint 3

5.4.2 Sprint 3 Business Logic

Action History

Every significant operation – status update, promise recorded, link generated, member invited – is written to a `CustomerActionHistory` table. Each entry stores the user ID, workspace ID, action type, target entity ID, and a timestamp. The history is exposed as a paginated, chronological table on the history screen, giving managers a complete audit trail of all workspace activity.

Dashboard Aggregation

Dashboard queries are computed server-side using Prisma `groupBy` queries. Debt records are grouped by campaign and status, returning counts for each combination. The frontend renders these counts as summary indicator cards and a per-campaign status breakdown. Using `groupBy` at the database level avoids fetching individual records and eliminates N+1 query patterns that would degrade performance on large campaigns.

Role-Based Permission Enforcement

Sensitive actions – deleting a campaign, modifying workspace settings, changing member roles – are protected by a role guard middleware. The guard reads the requesting user's workspace role from the database and rejects the request with `403 Forbidden` if the role is insufficient. Agents who attempt Manager-only operations receive a clear error response without any data leak.

5.4.3 Sprint 3 – Test Results

Table 5.4 presents the functional tests executed at the end of Sprint 3.

ID	Test Description	Expected Result	Actual
T3-01	Record a promise with a future date on a Notified debt	Status set to Promised, action logged	As expected
T3-02	Record a promise with a past date	Status immediately set to Overdue	As expected
T3-03	View action history as manager	Paginated chronological list displayed	As expected
T3-04	Open dashboard with active campaign data	KPI cards show correct debt counts	As expected
T3-05	Filter dashboard by specific campaign	Counts update to selected campaign only	As expected
T3-06	Agent attempts to delete a campaign	403 Forbidden, campaign unchanged	As expected
T3-07	Agent attempts to change a member's role	403 Forbidden, role unchanged	As expected
T3-08	Manager deletes a campaign (authorized action)	Campaign removed, history entry created	As expected

Table 5.4: Sprint 3 – Functional Test Results

5.4.4 Sprint 3 Retrospective

Dashboard aggregation and role-based access control required close coordination between database queries and route guards. The team standardized manager-only middleware checks to avoid duplicating permission logic in each handler. Stakeholder feedback during the review led to clearer campaign filters and more readable history entries for non-technical users.

Sprint 3 demonstrated that auditability features change user behaviour: agents record promises more consistently when the history timeline is visible during calls. The dashboard also became the default sprint-review screen for managers, which influenced Sprint 4 priorities (payment totals, invoice shortcuts, email tracking cards). Remaining risks before payment integration were stale dashboard caches after bulk imports; the team mitigated this by invalidating aggregation queries when import jobs complete.

Conclusion

Sprint 3 delivered action history logging, payment promise tracking, the manager dashboard, and role-based permission enforcement. The use of server-side `groupBy` aggregation was the key performance decision that kept the dashboard responsive even on workspaces with large debt portfolios. The following chapter presents Sprint 4, which completes the platform with payment confirmation, invoice generation, and email tracking analytics.

6 Sprint 4: Payment Confirmation, Invoicing, and Analytics Integration

Introduction

Sprint 4 consolidates the final operational flow from customer payment to invoice delivery while integrating per-debt multi-currency Stripe checkout and advanced email-tracking analytics. Its objective is to guarantee that every paid debt is confirmed safely, reflected in the platform state, and followed by invoice availability and auditable communication events. The most critical design challenge was hardening the Stripe payment return path to prevent duplicate charges while keeping customer feedback responsive during webhook processing delays.

6.1 Sprint 4 Backlog

Note: Sprint 4 tasks are expressed at the implementation level rather than as user stories, reflecting the integration and hardening nature of this sprint where the features involve system-to-system flows (Stripe, Brevo) rather than direct user-facing actions.

Tables 6.1 and 6.2 list the tasks for Sprint 4.

ID	Task	Period	Priority
S4-01	Implement Stripe checkout session creation from secure public debt links.	Weeks 9–10	High
S4-02	Add post-payment verification endpoint and frontend polling after Stripe redirect.	Weeks 9–10	High
S4-03	Enforce idempotent payment confirmation and duplicate session prevention.	Weeks 9–10	High
S4-04	Generate and expose invoices only after confirmed debt payment.	Weeks 9–10	High

Table 6.1: Sprint 4 Backlog (part 1)

6.2 Functional Specification

6.2.1 Use Case Diagram

Figure 6.1 shows the Sprint 4 use case diagram for payment confirmation, invoice access, and tracking analytics.

ID	Task	Period	Priority
S4-05	Send payment receipt email with invoice access link through Brevo.	Weeks 9–10	Medium
S4-06	Persist and aggregate email-tracking events (sent, clicked, spam-like signals).	Weeks 9–10	High
S4-07	Finalize manager statistics view and campaign-level tracking synchronization.	Weeks 9–10	Medium
S4-08	Store per-debt ISO currency and pass it to Stripe Checkout and hosted invoices.	Weeks 9–10	High

Table 6.2: Sprint 4 Backlog (part 2)

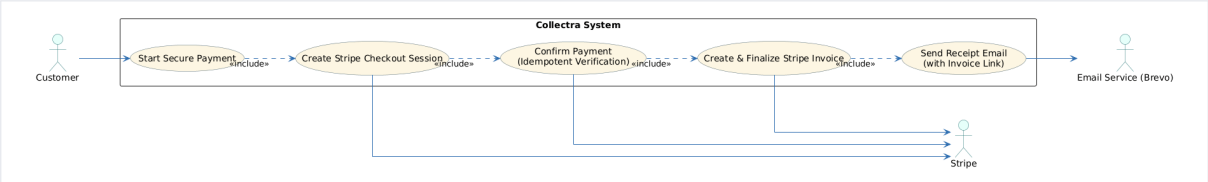

Figure 6.1: Use Case Diagram – Sprint 4 Payment and Invoice Flow

6.2.2 Textual Description of Use Cases

Sprint 4 introduces two final user-facing use cases:

- **UC-09: Confirm Debt Payment and Issue Invoice** – the system confirms payment (webhook and/or explicit verification), marks the debt as paid, generates an invoice number, and allows secure invoice access through the public token;
- **UC-10: Monitor Communication and Campaign Tracking** – managers consult campaign email statistics and synchronized tracking indicators (sent, clicked, spam-like events) to supervise collection effectiveness.

6.3 Design

6.3.1 Class Diagram

Figure 6.2 shows the Sprint 4 class diagram, integrating payment and invoice-related fields (invoice number, pending Stripe session lock) together with tracking entities used by analytics endpoints.

6.3.2 Sequence Diagrams

6.4 Implementation

6.4.1 Sprint 4 Interface

Sprint 4 introduces post-payment customer feedback, invoice access, and enhanced manager tracking cards synchronized with backend event ingestion.

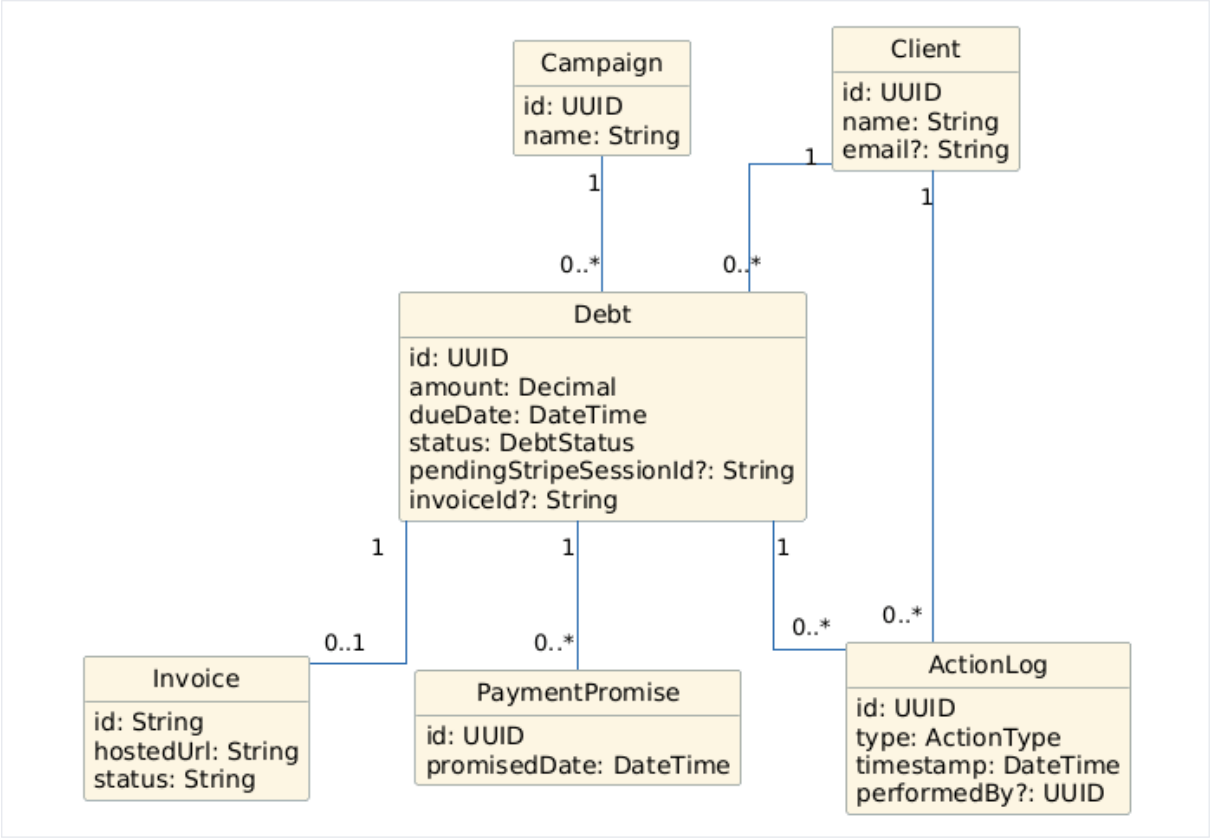

Figure 6.2: Class Diagram – Sprint 4 Payment, Invoice, and Tracking Layer

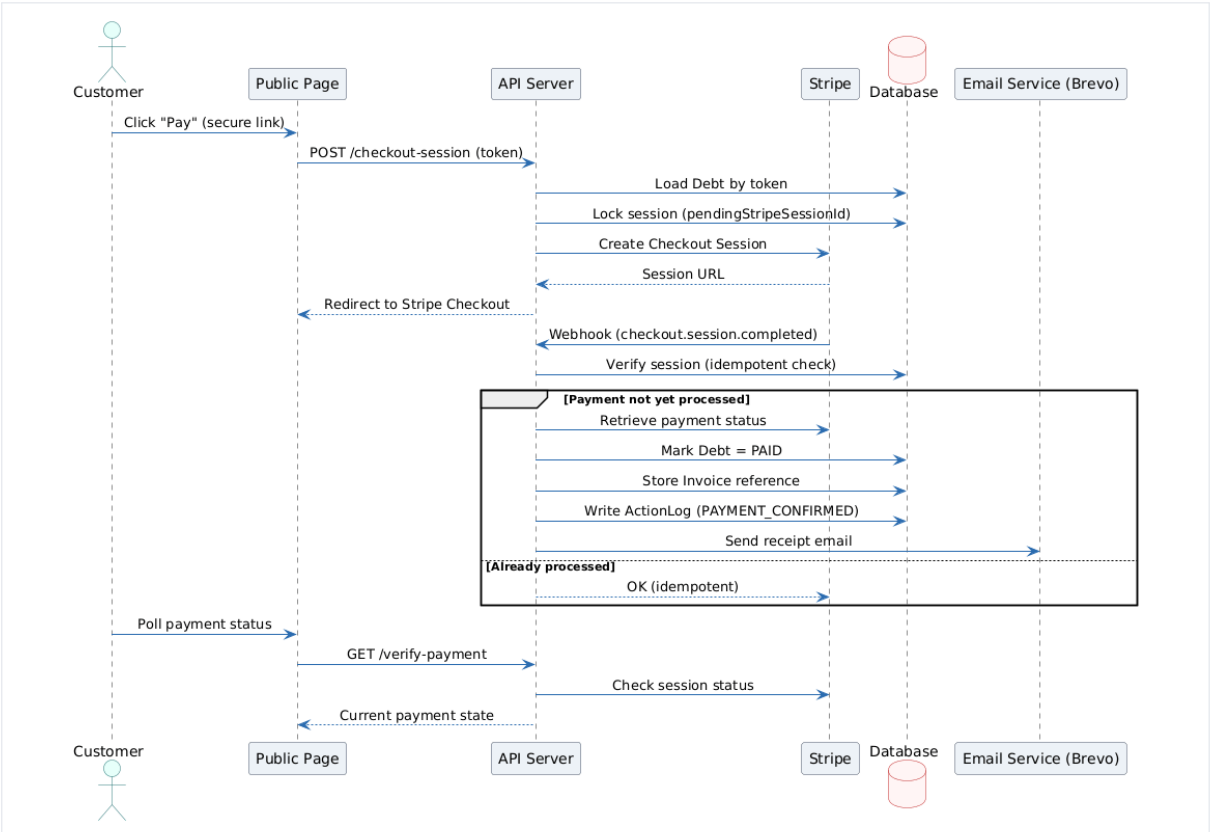

Figure 6.3: Sequence Diagram – Stripe Verification and Payment Confirmation

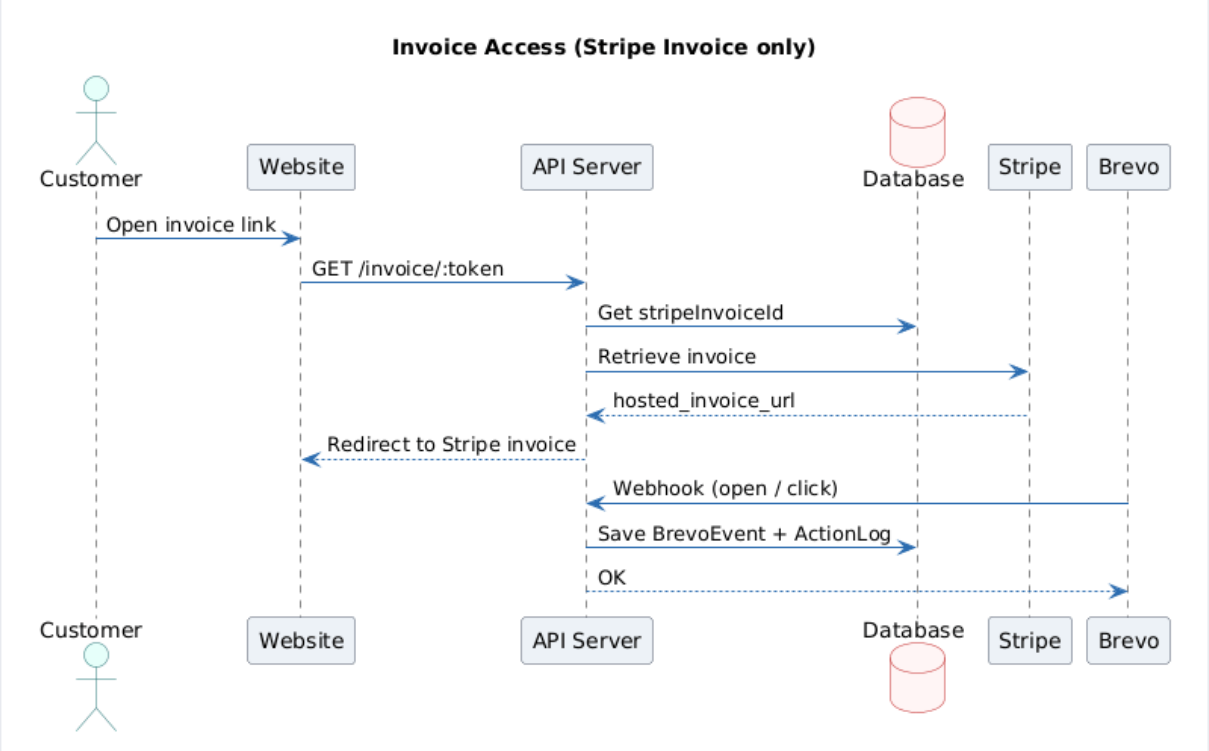

Figure 6.4: Sequence Diagram – Invoice Access

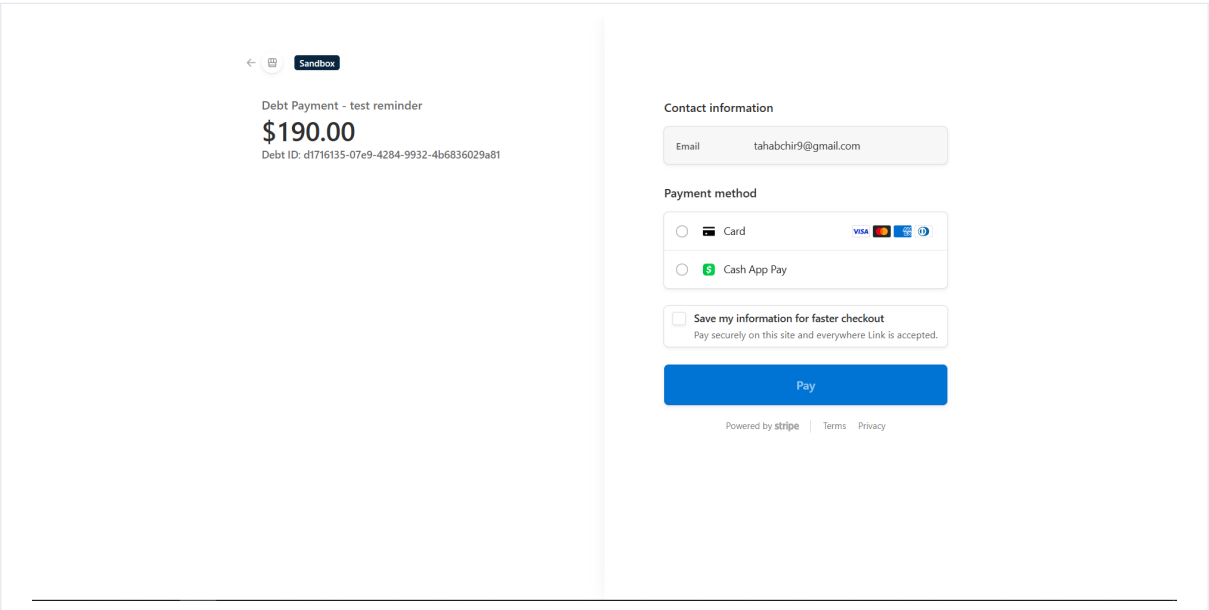

Figure 6.5: Payment and Tracking Overview – Sprint 4

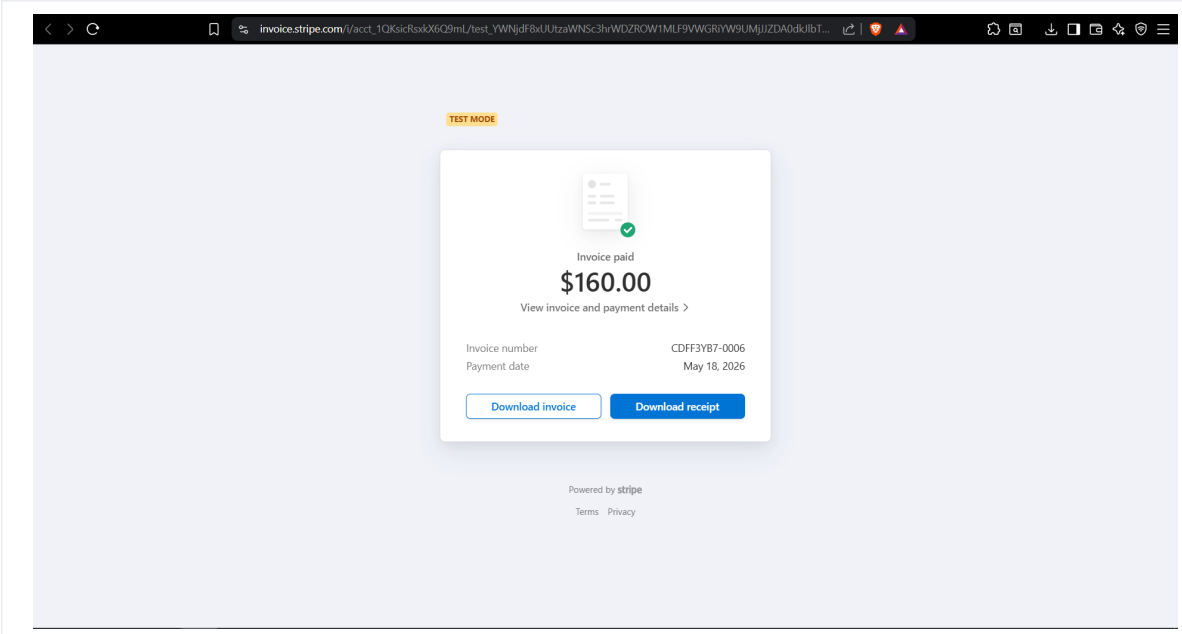

Figure 6.6: Invoice and Analytics Areas – Sprint 4

6.4.2 Sprint 4 Business Logic

Payment Confirmation and Idempotency

The payment pipeline was hardened to prevent duplicate charges and stale frontend states. After Stripe redirection, the client calls a verification endpoint and polls until the database confirms the debt as `PAID`. Server-side confirmation is idempotent: repeated webhook or verification calls do not duplicate state transitions.

The backend enforces the following safeguards:

- a `pendingStripeSessionId` lock to block concurrent checkout sessions;
- strict eligibility checks before payment start (status and promise-date rules);
- idempotent confirmation that updates debt status only once.

This prevents duplicate payment initiation and keeps User Interface state aligned with persisted backend data.

Invoice Generation and Delivery

Invoice generation occurs only after confirmed payment. When the debt transitions to `PAID`, the service creates or reuses a stable invoice number, exposes a secure public invoice endpoint, and redirects users to the hosted invoice or printable Portable Document Format representation.

Invoice email is sent as a post-confirmation side effect through Brevo, ensuring payment state remains the primary source of truth even if email delivery fails.

Multi-Currency Debts and Stripe Checkout

Each debt record stores an ISO 4217 currency code (three letters, default `usd`) in the `currency` column added during Sprint 4. Agents and managers can set currency when creating or updating a debt, and bulk Comma-Separated Values imports accept a currency column so an entire campaign batch can be created in euros, dollars, or another supported code.

The backend normalizes currency values through `normalizeStripeCurrency()`, which falls back to the workspace default `STRIPE_CURRENCY` environment variable when a value is missing. Stripe Checkout uses the debt currency for `price_data.currency` and converts the decimal amount to the smallest currency unit (`unit_amount = round(amount × 100)` for two-decimal currencies such as USD and EUR). Hosted invoices and idempotency keys are scoped per debt and currency so retries do not create duplicate billing artifacts.

This design allows international collection scenarios without changing the core payment confirmation pipeline: webhooks still map sessions to debts through metadata, and status transitions remain idempotent regardless of currency.

Email Tracking and Manager Analytics Synchronization

Campaign import and communication flows now persist normalized tracking actions (`EMAIL_SENT`, `LINK_CLICKED`, and mapped provider events) into action history. Manager endpoints aggregate these events into operational metrics and refresh them in near real time.

The final dashboard therefore combines:

- debt lifecycle indicators;
- payment and invoice completion visibility;
- communication performance metrics for campaign follow-up decisions.

6.4.3 Sprint 4 – Test Results

Table 6.3 presents the functional tests executed at the end of Sprint 4.

6.4.4 Sprint 4 Retrospective

Payment idempotency and invoice side effects were the most sensitive topics. Testing focused on duplicate verification calls and email failures after successful payment. The retrospective emphasized maintaining payment state as the source of truth while treating notifications as best-effort operations.

Stripe test mode and controlled demo confirmation allowed end-to-end rehearsals without touching live card data. Invoice generation was simplified to a stable public route so support staff can resend links without re-running payment. Analytics cards for Brevo events were scoped as diagnostic aids rather than billing systems, which kept the scope realistic for a twelve-week internship. The final retrospective concluded that Collectra is deployable as a

ID	Test Description	Expected Result	Actual
T4-01	Create Stripe checkout session for an eligible debt	Session Uniform Resource Locator returned, lock set on debt	As expected
T4-02	Create second checkout session while lock is active	409 Conflict, duplicate session blocked	As expected
T4-03	Verify payment after successful Stripe redirect	Debt status set to PAID, lock cleared	As expected
T4-04	Call verify-payment endpoint twice (idempotency check)	Second call returns PAID without re-processing	As expected
T4-05	Access invoice endpoint after confirmed payment	Invoice page displayed or Portable Document Format accessible	As expected
T4-06	Access invoice endpoint before payment confirmation	403 Forbidden, no invoice served	As expected
T4-07	Payment receipt email sent via Brevo after confirmation	Email received with invoice link	As expected
T4-08	Tracking pixel triggered on customer page open	EMAIL_SENT event persisted in history	As expected
T4-09	Manager views campaign email statistics	Aggregated sent/clicked counts displayed	As expected
T4-10	Sync Brevo provider events via sync endpoint	New events merged into action history	As expected
T4-11	Pay debt in non-default currency (e.g. EUR) via Stripe	Checkout uses debt currency; debt marked PAID	As expected

Table 6.3: Sprint 4 – Functional Test Results

minimum viable product for small collection teams, with future work on advanced dunning rules and richer currency reporting.

6.5 Integrated Final Architecture

Figures 6.8 and 6.7 present the overall technical architecture diagrams shows the fully integrated class diagram combining all entities and relationships across all four sprints.

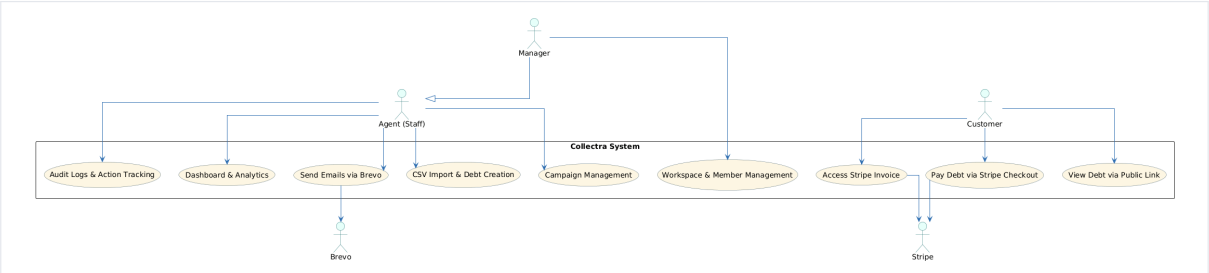

Figure 6.7: Use Case Diagram – Overall Technical Architecture

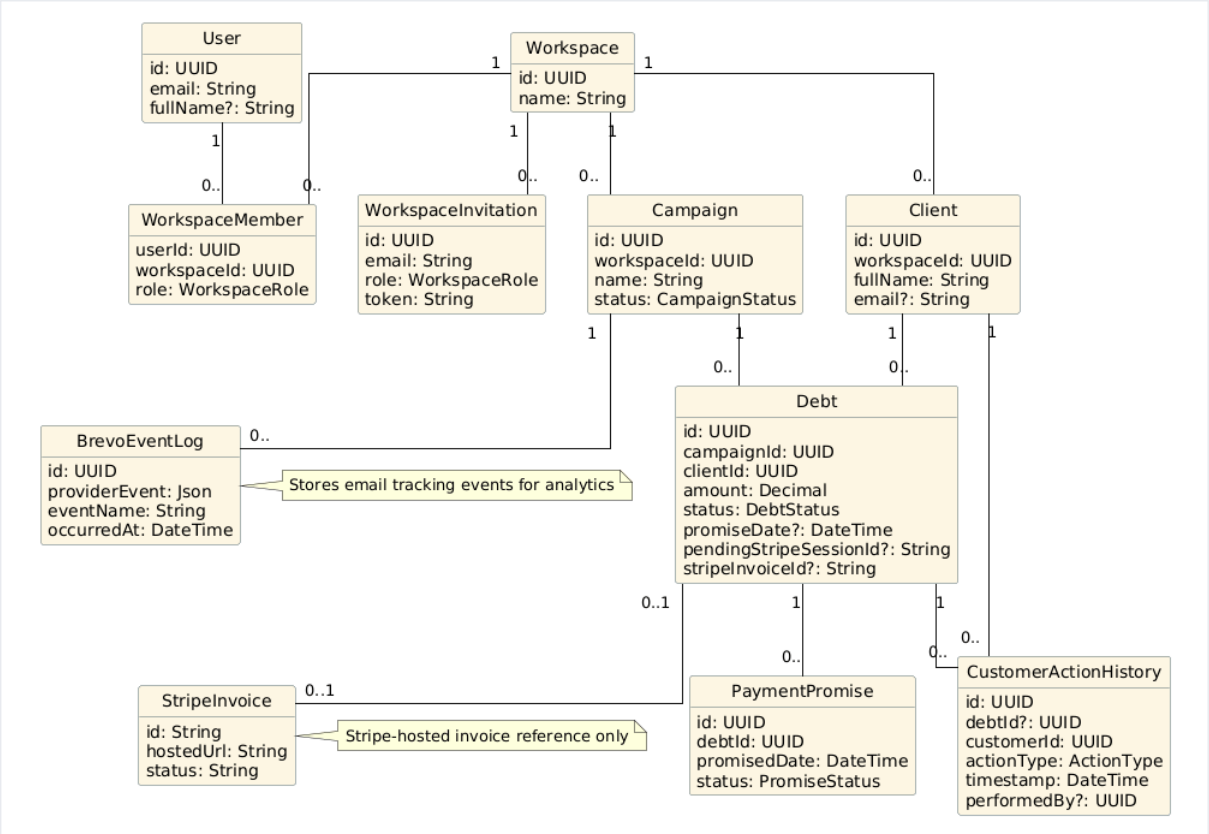

Figure 6.8: Class Diagram – Overall Technical Architecture

Conclusion

Sprint 4 delivered a complete and hardened post-payment workflow: Stripe verification, per-debt currency handling, idempotent payment confirmation, secure invoice generation, and invoice email delivery. It also finalized communication tracking integration for manager analytics, resulting in an end-to-end operational flow from customer action to auditable reporting. The idempotent confirmation design – using a database-level session lock combined with a polling verification endpoint – was the most technically demanding solution of the project and directly addresses the risk of double-charging customers.

7 Cross-Cutting Technical Topics

Introduction

While sprint chapters follow delivery order, several mechanisms span the entire platform. This chapter summarises the email pipeline, payment and invoice flow, the in-app support chatbot, error handling philosophy, and operational practices that support maintainability in production.

7.1 In-App Support Chatbot

Collectra includes a floating **support chatbot** available to authenticated managers and agents from the dashboard layout. The widget opens a panel where users type questions about campaigns, Comma-Separated Values import, overview statistics, account settings, payment links, and invoices.

The web application sends each question to `POST /api/support/chat`, a Next.js route that calls the **OpenRouter** API with a fixed system prompt describing Collectra capabilities. The default model is `deepseek/deepseek-chat-v3-0324`; configuration uses `OPENROUTER_API_KEY`, `OPENROUTER_API_BASE_URL`, and related environment variables.

When the remote model is unavailable or returns an error, the client falls back to a **rule-based assistant** implemented in `SupportChatbot`: keyword patterns suggest relevant answers and deep links (for example `/campaigns`, `/overview`, `/settings/account`). Quick-reply chips and a reset action keep the experience usable without reloading the page.

The chatbot is intentionally scoped as **operational guidance**, not autonomous debt processing: it does not modify database records or trigger payments. It reduces onboarding friction for new agents and documents common workflows referenced in this report.

7.2 Email and Notification Pipeline

Collectra uses Brevo for transactional email and event tracking. When agents trigger customer communications, the platform records outbound messages and synchronises provider events (`EMAIL_SENT`, `LINK_CLICKED`, opens) into action history for manager review. The design keeps email diagnostics separate from core debt state: a failed provider callback does not corrupt payment status, but the event log helps support teams investigate delivery issues.

7.3 Payment, Invoice, and Customer Return Flow

The customer journey on a personal link follows a controlled sequence: view debt details, optionally submit a promise-to-pay date, start Stripe Checkout, return to a verification endpoint, and access a hosted invoice once payment is confirmed. Server-side locks prevent parallel checkout sessions for the same debt. Verification is idempotent: repeated calls after success return the same final state without duplicate charges or duplicate history entries.

7.4 Error Handling and User Feedback

Application Programming Interface errors return structured JSON payloads with stable error codes. The frontend maps these codes to actionable messages (invalid import row, expired link, forbidden workspace access). For long-running imports, the backend returns per-row skip reasons so agents can correct files instead of failing the entire batch silently.

7.5 Performance Considerations

Large Comma-Separated Values files are processed in chunks inside database transactions. Dashboard indicators rely on aggregation queries rather than per-debt loops. Preview deployments on Vercel allowed measuring import duration and dashboard load time before promoting changes to production.

7.6 Documentation and Knowledge Transfer

OpenAPI documentation is generated from Zod schemas, keeping the contract close to runtime validation. Sprint reviews at CyberBridge served as living documentation: each demo explained assumptions, limitations, and follow-up tasks recorded in Linear for continuity after the internship.

7.7 Observability and Operational Monitoring

Although full observability stacks were out of scope, the team adopted lightweight practices suitable for a Vercel-hosted monorepo. Build logs and deployment previews on Vercel provide the first line of diagnosis when a merge fails. Application Programming Interface routes log structured errors server-side (validation code, workspace identifier, route name) without leaking secrets to clients. For payment flows, verification endpoints return explicit states (`pending`, `confirmed`, `failed`) so the frontend can stop polling deterministically. This approach supported internship-scale operations while leaving room to integrate tools such as Sentry or Logtail in a future iteration.

7.8 Data Protection and Privacy Considerations

Collectra stores personal data (customer emails, debt amounts, communication history). Workspace isolation ensures agents never access another tenant's records. Customer links are time-limited and signed; expired tokens return `TOKEN_EXPIRED` without revealing whether the identifier ever existed. Hypertext Transfer Protocol-only cookies protect session tokens from cross-site scripting in the browser. These measures align with common Software as a Service expectations and were reviewed with the company tutor during Sprint 4.

7.9 Future Evolution Roadmap

Table 7.1 lists enhancements discussed during retrospectives but intentionally deferred to keep the internship deliverable focused.

Enhancement	Priority	Rationale
Fine-grained agent permissions per campaign	Medium	Larger teams need delegation
Exportable compliance reports (Portable Document Format / Excel)	High	Manager audits
Webhook signature hardening dashboard	Low	Operational security polish

Table 7.1: Collectra Evolution Roadmap (Post-Internship)

Conclusion

These cross-cutting topics complement the sprint-by-sprint narrative. They highlight how authentication, imports, payments, and communications remain coherent when implemented as platform-wide rules rather than isolated features.

General Conclusion

This report presented the full design and implementation of **Collectra**, a web platform for debt collection and campaign management, developed during a final-year internship at CyberBridge .

Summary of achievements. The project successfully delivered a complete, production-ready Software as a Service application across four Scrum sprints. Sprint 1 established the authentication system and workspace infrastructure. Sprint 2 introduced campaign management, bulk Comma-Separated Values import, a full debt lifecycle, and JSON Web Token-secured customer personal links. Sprint 3 added action history logging, payment promise recording, and the operational manager dashboard. Sprint 4 completed the payment-to-invoice pipeline with Stripe verification, per-debt multi-currency checkout, idempotent confirmation, secure invoice access, and synchronized campaign tracking analytics. An in-app support chatbot assists managers and agents throughout the dashboard. The platform is live on Vercel at `collectra.xyz`, deployed automatically via a GitHub-based Continuous Integration and Continuous Deployment pipeline.

Non-functional requirements validation. Looking back at the requirements defined in Chapter 2, all five non-functional requirements were met:

- **Security:** JSON Web Tokens are signed with Hash-based Message Authentication Code SHA-256, stored in Hypertext Transfer Protocol-only cookies, and verified by middleware on every protected route. Workspace isolation is enforced at every layer, from database queries to Application Programming Interface middleware to frontend navigation.
- **Maintainability:** the monorepo structure with a single shared Prisma schema and shared Zod types means that any model change propagates consistently across frontend and backend without manual synchronization.
- **Performance:** the Comma-Separated Values import uses Prisma transaction batching to avoid per-row round trips, and the dashboard uses `groupBy` aggregation to prevent N+1 queries on large campaign data sets.
- **Reliability:** Zod validates every Application Programming Interface input at runtime, and the idempotent payment confirmation logic prevents duplicate state transitions regardless of how many times the verify endpoint is called.
- **Usability:** iterative sprint reviews with CyberBridge stakeholders confirmed that the workflow from campaign creation to payment confirmation requires minimal training for new collection agents.

Technical contributions. From a technical perspective, the project demonstrates a full-stack TypeScript monorepo architecture combining Next.js, Hono, Prisma, and Supabase in a production-grade setup. The use of Zod for both runtime validation and OpenAPI schema generation, combined with workspace-level data isolation enforced at every layer, shows how security and maintainability can be built in from the very start of a project. The final implementation also demonstrates robust financial-flow design through duplicate-session prevention, idempotent confirmation logic, and post-transaction invoice and email side effects.

Challenges encountered. The main challenges faced during the project were:

- designing a flexible but strict debt status transition system that prevents invalid lifecycle jumps while remaining easy to extend;
- handling large Comma-Separated Values files efficiently without overloading the database or causing request timeouts;
- securing the Stripe payment return path to avoid duplicate payments while keeping customer feedback responsive during webhook delays;
- ensuring correct workspace isolation at every layer of the application, from database queries to Application Programming Interface middleware to frontend navigation.

Each challenge was resolved through careful schema design, server-side validation, and iterative testing with real data.

Personal learning outcomes. This internship provided hands-on experience in full-stack TypeScript development, agile project management with Scrum, and professional software delivery in a real company context. It deepened my understanding of authentication flows, role-based access control, JSON Web Token-based security, real-world relational data modeling, and production deployment with Vercel and GitHub Continuous Integration and Continuous Deployment pipelines.

Perspectives and future work. Several improvements could extend the Collectra platform in the future:

- integration with SMS or WhatsApp for direct, automated customer notifications;
- a scheduled background job for automatic overdue debt detection;
- campaign report export in Portable Document Format or Excel format;
- a mobile-responsive redesign or dedicated mobile application;
- advanced analytics and collection performance reporting for managers.

In conclusion, Collectra meets all the objectives set at the beginning of the internship. It provides collection teams with a secure, structured, and easy-to-use platform that replaces fragmented tools with a single, purpose-built solution for debt management.

Reference List

[1] Schwaber, K. and Sutherland, J. (2020). *The Scrum Guide – The Definitive Guide to Scrum: The Rules of the Game*. Scrum.org. Available at: https://scrumguides.org

[2] Vercel. *Next.js Documentation*. Available at: https://nextjs.org/docs (Accessed: 2026)

[3] Meta Open Source. *React Documentation*. Available at: https://react.dev (Accessed: 2026)

[4] Microsoft. *TypeScript Documentation*. Available at: https://www.typescriptlang.org/docs (Accessed: 2026)

[5] Prisma. *Prisma Object-Relational Mapper Documentation*. Available at: https://www.prisma.io/docs (Accessed: 2026)

[6] Supabase, Inc. *Supabase Documentation*. Available at: https://supabase.com/docs (Accessed: 2026)

[7] Hono. *Hono – Ultrafast Web Framework for the Edges*. Available at: https://hono.dev/docs (Accessed: 2026)

[8] Vercel. *Turborepo Documentation*. Available at: https://turbo.build/repo/docs (Accessed: 2026)

[9] Tailwind Labs. *Tailwind CSS Documentation*. Available at: https://tailwindcss.com/docs (Accessed: 2026)

[10] Stripe, Inc. *Stripe API Reference*. Available at: https://stripe.com/docs/api (Accessed: 2026)

[11] Brevo (formerly Sendinblue). *Brevo API Documentation*. Available at: https://developers.brevo.com (Accessed: 2026)

[12] Vercel. *Vercel Deployment Documentation*. Available at: https://vercel.com/docs (Accessed: 2026)

[13] Jones, M., Bradley, J., and Sakimura, N. (2015). *JSON Web Token – Introduction*. RFC 7519. IETF. Available at: https://datatracker.ietf.org/doc/html/rfc7519

[14] OpenAPI Initiative. (2023). *OpenAPI Specification v3.1.0*. Available at: https://spec.openapis.org/oas/v3.1.0

[15] Colinhacks. *Zod – TypeScript-first Schema Validation*. Available at: https://zod.dev (Accessed: 2026)

A Domain Glossary

This glossary defines recurring business and technical terms used in the report.

Workspace Isolated tenant environment containing campaigns, debts, members, and history.

Campaign Collection operation grouping debts that share communication and reporting context.

Debt Individual amount owed by a customer, with lifecycle status and payment metadata.

Personal link Time-limited, signed Uniform Resource Locator allowing a customer to view or act on a debt without an account.

Promise to pay Customer commitment date recorded by an agent or submitted through the public link.

Action history Chronological log of operational events (status updates, promises, payments, tracking).

Idempotent confirmation
Payment verification that yields the same final state when executed multiple times.

Import summary
Structured result of a Comma-Separated Values upload (imported, skipped, error reasons).

Manager dashboard
Aggregated indicators (counts by status, campaign filters) for supervision.

Preview deployment
Temporary Vercel build used to validate a pull request before production merge.

Representational State Transfer (REST)
Architectural style used by the Hono Application Programming Interface;

resources are addressed by nouns (`/debts`, `/campaigns`) and manipulated with standard HTTP verbs.

Zod schema TypeScript-first validation definition shared between backend handlers and OpenAPI generation.

Stripe Checkout
Hosted payment page redirecting customers to a secure card entry flow without storing card data in Collectra.

Webhook Server-to-server callback (Stripe, Brevo) notifying Collectra of external events.

Tenant Synonym for workspace in multi-tenant Software as a Service terminology.

Dunning Systematic process of communicating with customers to recover overdue debts.

Hypertext Transfer Protocol-only (HTTP-only) cookie
Session cookie not accessible to JavaScript, mitigating cross-site scripting token theft.

B Application Interface Walkthrough

This appendix presents a screen-by-screen walkthrough of the Collectra user interface. Each subsection explains the purpose of the screen, the primary user actions, and how the screen connects to the backend modules described in the sprint chapters. Figures are reproduced here at appendix scale for quick visual reference during the defense.

B.1 Sprint 1 – Authentication and Workspace

B.1.1 Login Screen

The login screen is the entry point for managers and agents. It validates credentials through Supabase Auth and, on success, receives a signed JSON Web Token stored in a Hypertext Transfer Protocol-only cookie. From a usability perspective, the layout keeps the form minimal (email, password, navigation to registration) so first-time users are not distracted by workspace features they cannot access yet.

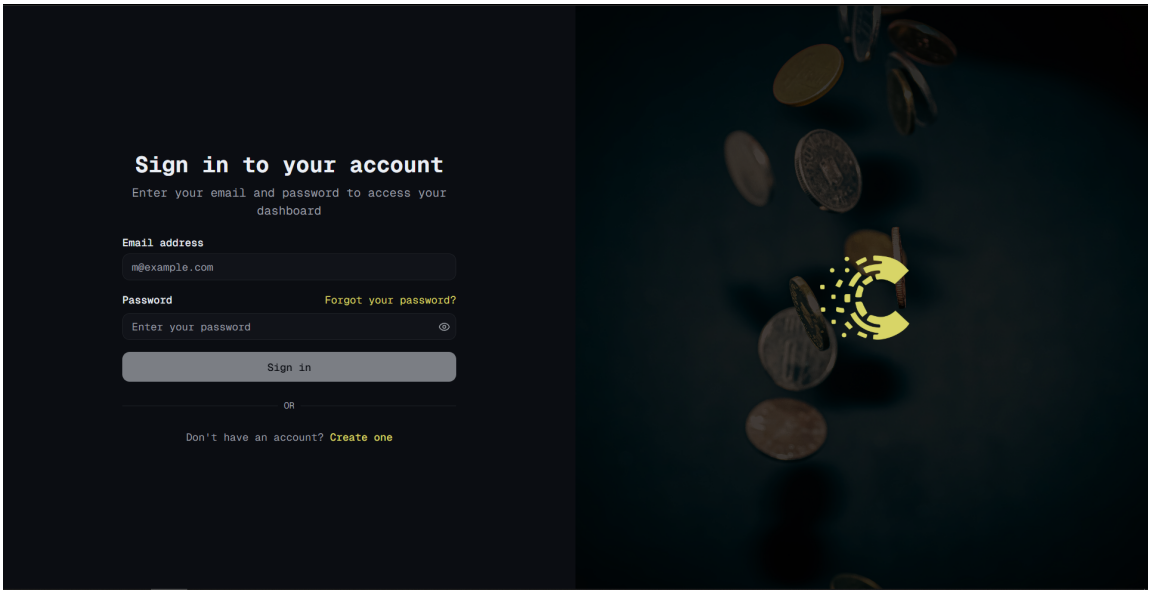

Figure B.1: Login Screen – Appendix View

B.1.2 Workspace Creation

After authentication, a user without membership is guided to create or join a workspace. Workspace creation defines the tenant boundary used for all later campaigns, debts, and history. The creator receives the manager role automatically, which is required for invitations and dashboard access in later sprints.

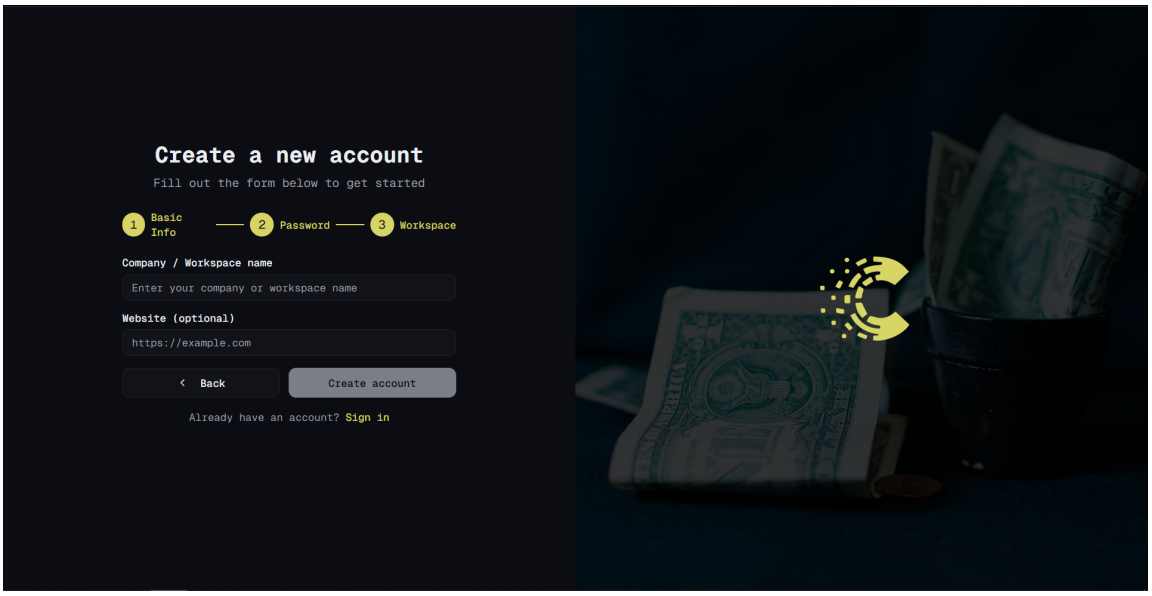

Figure B.2: Workspace Creation – Appendix View

B.1.3 Team Management and Invitations

The team screen lists current members and pending invitations. Managers can invite agents by email and assign roles before the invitee accepts. This screen demonstrates the role-based access control foundation implemented in Sprint 1.

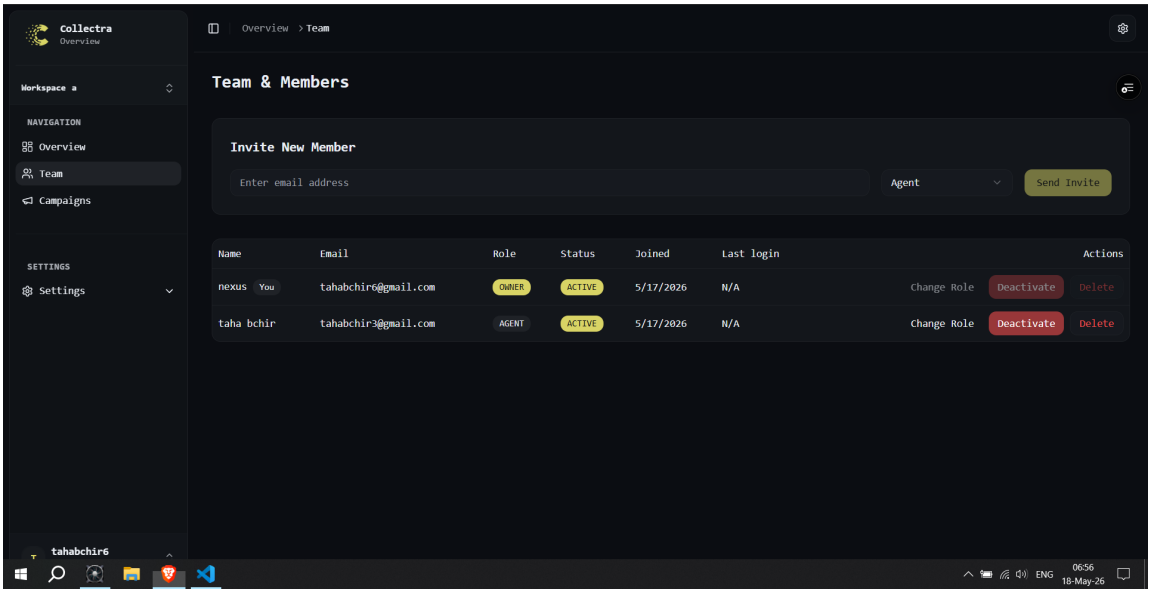

Figure B.3: Team Management – Appendix View

B.2 Sprint 2 – Campaign and Debt Management

Sprint 2 screens introduce the operational core of Collectra: structuring work into campaigns, loading debts at scale, and exposing customer-facing links. Agents spend most of their time in

the debt table; managers periodically review campaign lists to balance workload across teams. The following figures highlight filtering, import feedback, and the minimal public link layout optimised for mobile customers.

B.2.1 Campaign List

The campaign list is the operational home for agents and managers inside a workspace. Each row summarizes campaign metadata and provides entry points to debt tables and import tools. The layout prioritizes quick navigation because agents switch campaigns frequently during a workday.

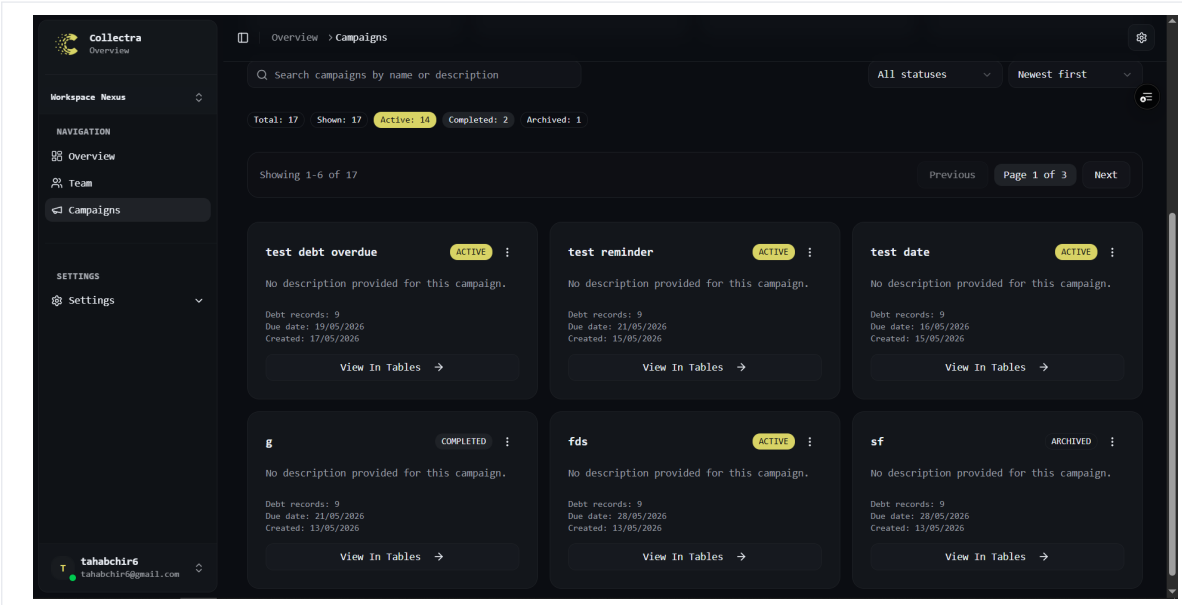

Figure B.4: Campaign List – Appendix View

B.2.2 Debt Table with Status Filters

The debt table is the core working surface for collection agents. Status filters align with the lifecycle model (imported, notified, promised, paid, overdue). Agents update statuses, open detail drawers, and generate customer links without leaving the table.

B.2.3 Comma-Separated Values Import Dialog

The import dialog accepts bulk files and displays a structured summary after upload. Agents can see how many rows were imported, skipped, or rejected with reasons. This screen validates the tolerant parser and chunked insert strategy described in Chapter 4.

B.2.4 Customer Personal Link Screen

Agents generate time-limited personal links from the debt context. The screen shows link status and expiration so agents can confirm the customer received a valid Uniform Resource Locator before closing the case. The same flow underpins email campaigns tracked in Sprint 4.

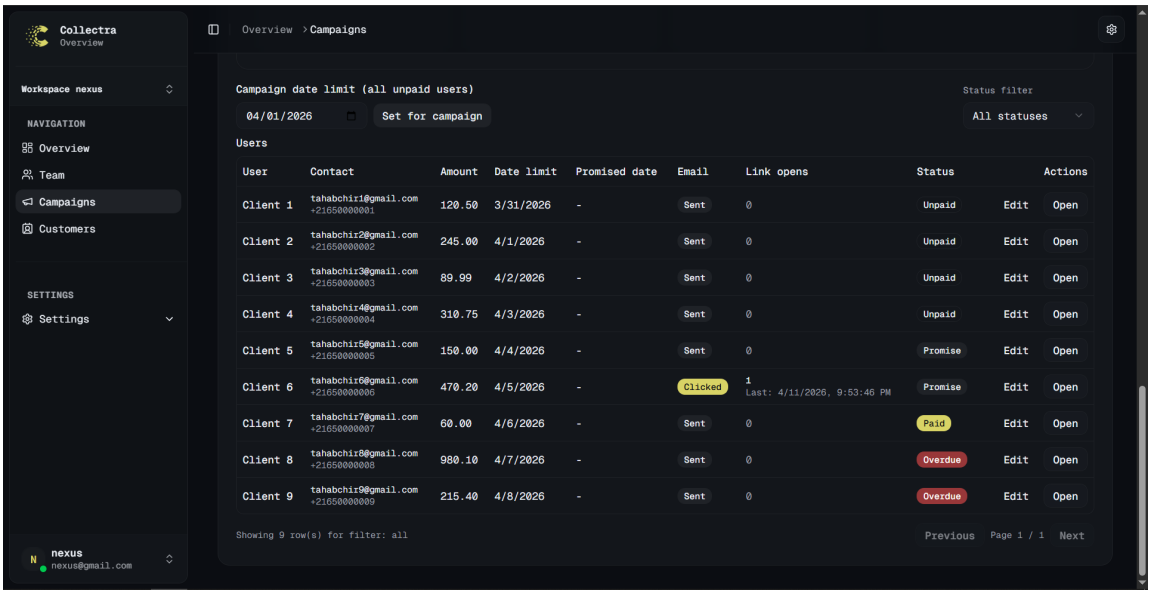

Figure B.5: Debt Table – Appendix View

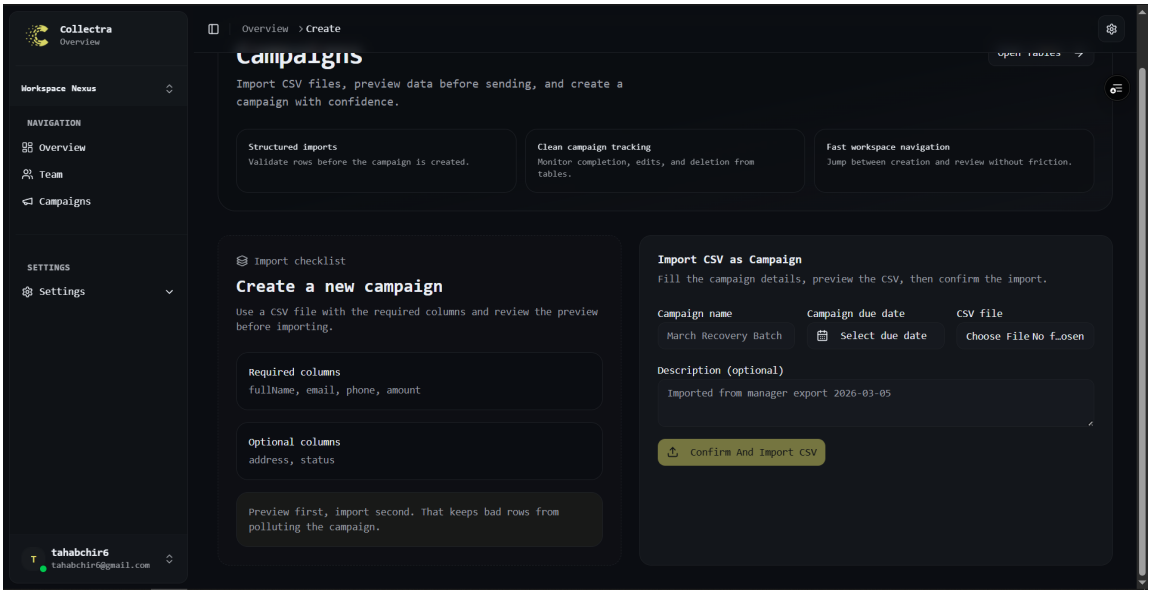

Figure B.6: Import Dialog – Appendix View

B.3 Sprint 3 – Collaboration and Reporting

Sprint 3 shifts the product from transactional updates to supervisory visibility. Managers need trustworthy aggregates; agents need an audit trail that survives staff turnover. The screens below show how promises, history, and dashboards interlock: every promise creates history, and dashboard totals react to status changes without manual refresh hacks.

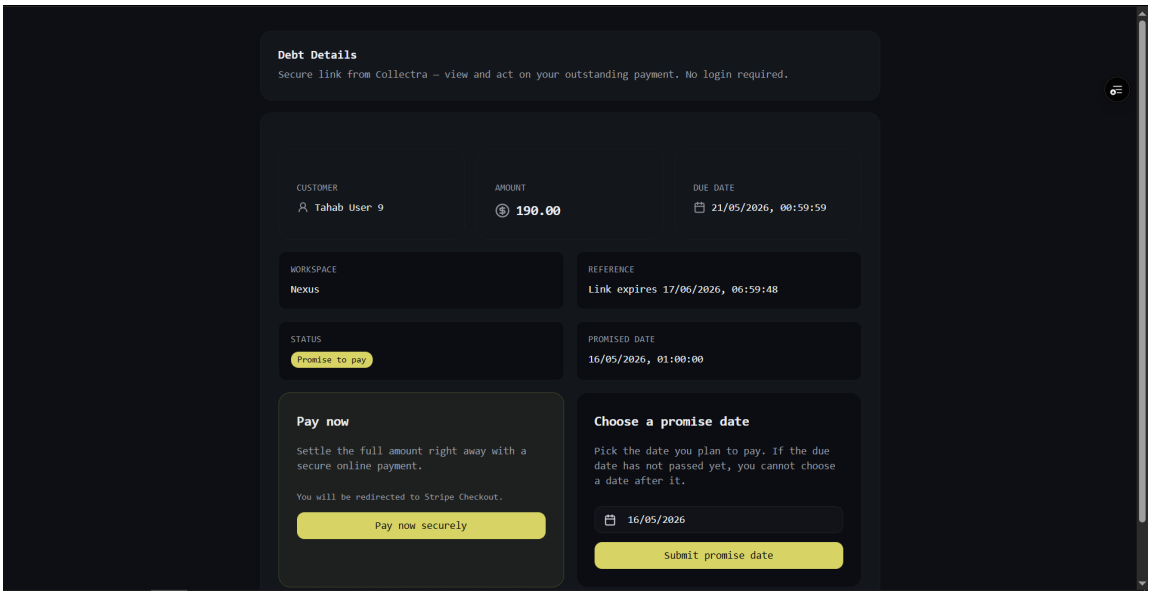

Figure B.7: Personal Link Screen – Appendix View

B.3.1 Payment Promise Dialog

Agents record promise-to-pay dates when a customer commits to a future payment. The dialog writes to `PaymentPromise` and updates debt status according to business rules. Managers later see the promise in history and dashboard indicators.

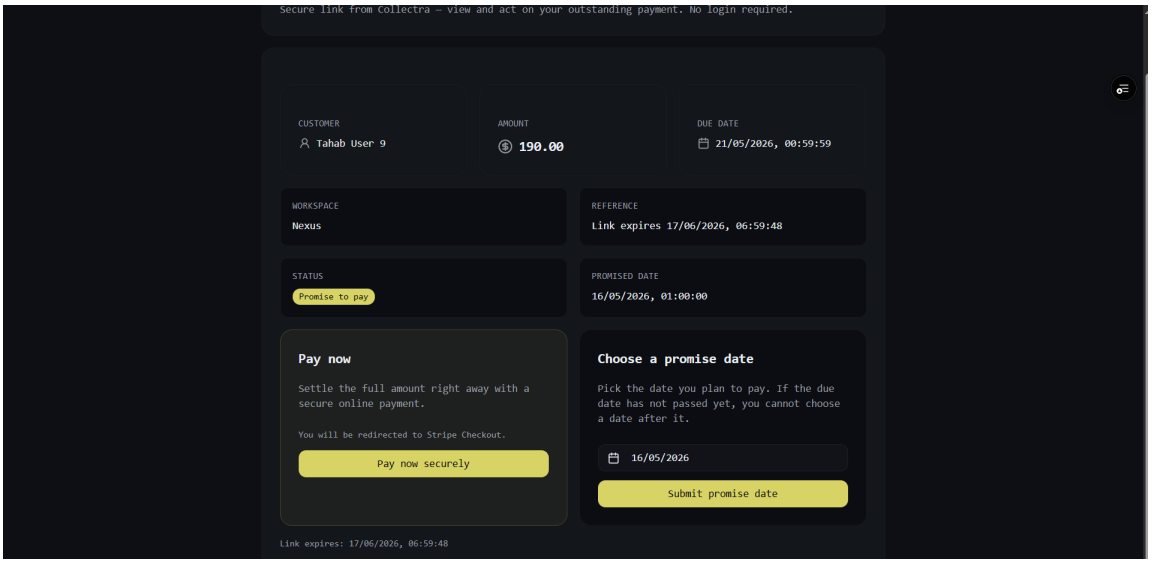

Figure B.8: Payment Promise Dialog – Appendix View

B.3.2 Action History Table

The history table provides chronological traceability across the workspace. Each row corresponds to a business event (status change, promise, payment, tracking signal). This screen answers audit questions that spreadsheets cannot handle reliably.

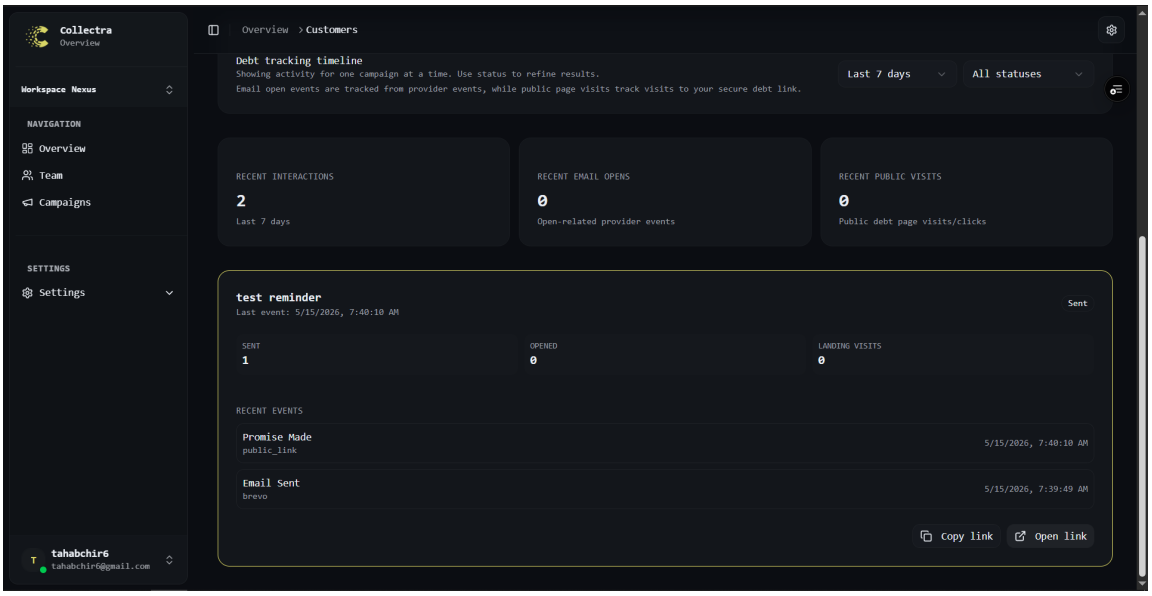

Figure B.9: Action History – Appendix View

B.3.3 Manager Dashboard

The dashboard aggregates counts by debt status and supports campaign filtering. It is manager-only and relies on server-side aggregation to stay responsive on large portfolios. Stakeholders used this screen during sprint reviews to validate reporting value.

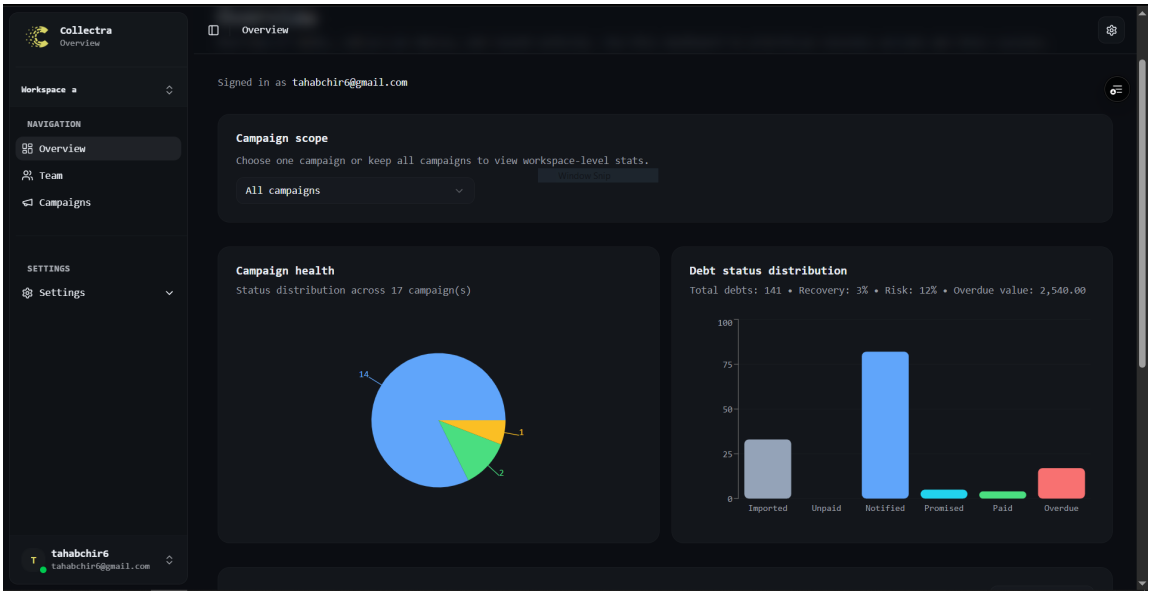

Figure B.10: Dashboard – Appendix View

B.4 Sprint 4 – Payment, Invoicing, and Analytics

Sprint 4 closes the business loop: a notified debt can become a paid debt with provable invoice artefacts. The user interface emphasises clarity after checkout—customers must understand

verification states, while managers need confidence that analytics reflect real provider events. These screens were validated with Stripe test cards and Brevo sandbox credentials.

B.4.1 Payment and Tracking Overview

Sprint 4 extends the manager view with payment-oriented indicators and communication metrics. The overview connects collection progress with email engagement signals synchronized from Brevo. It supports operational decisions such as prioritizing campaigns with low open rates.

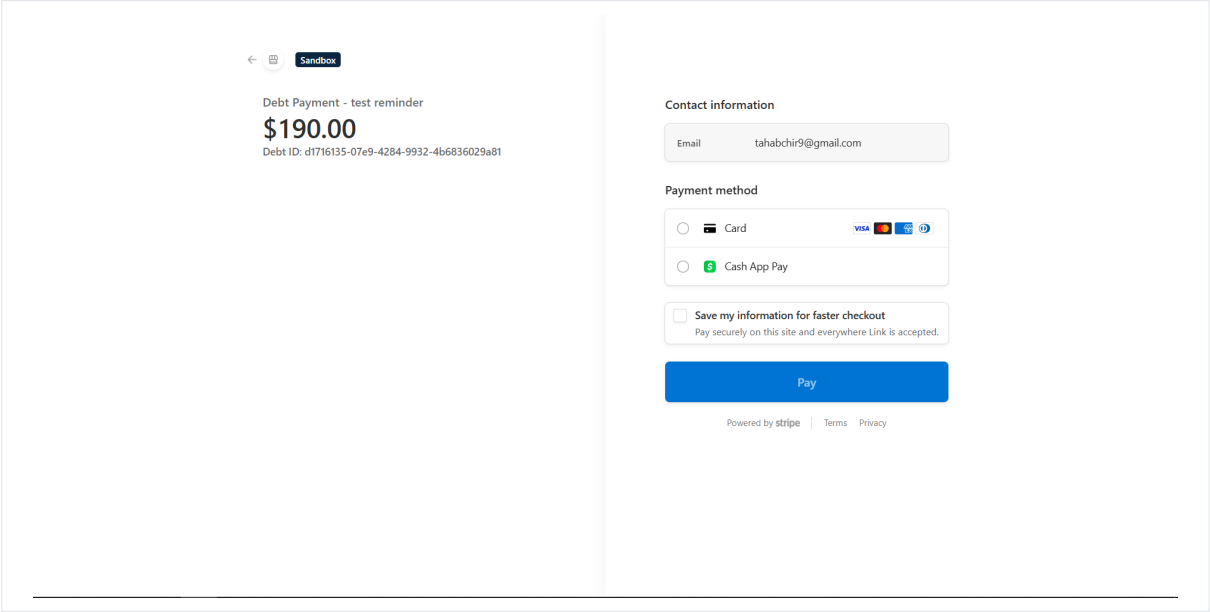

Figure B.11: Payment Overview – Appendix View

B.4.2 Invoice and Analytics Areas

The invoice area becomes available after confirmed payment. Customers can access a hosted invoice; managers can verify that payment, history, and tracking states remain consistent. Empty or low-activity states are handled explicitly so users understand when analytics are not yet available.

B.4.3 Support Chatbot

The floating support widget is available on authenticated dashboard pages. Users open the panel, ask questions in natural language, and receive answers from the OpenRouter-backed assistant or from the built-in keyword fallback when the remote model is unavailable. Suggested quick links route agents to campaigns, overview, and account settings.

B.5 Guidance for Additional Screenshots

To further enrich this appendix, you may add images under `screenshots/` or `screennshots/` and reference them with the `\screenshot` macro. Recommended captures include the regis-

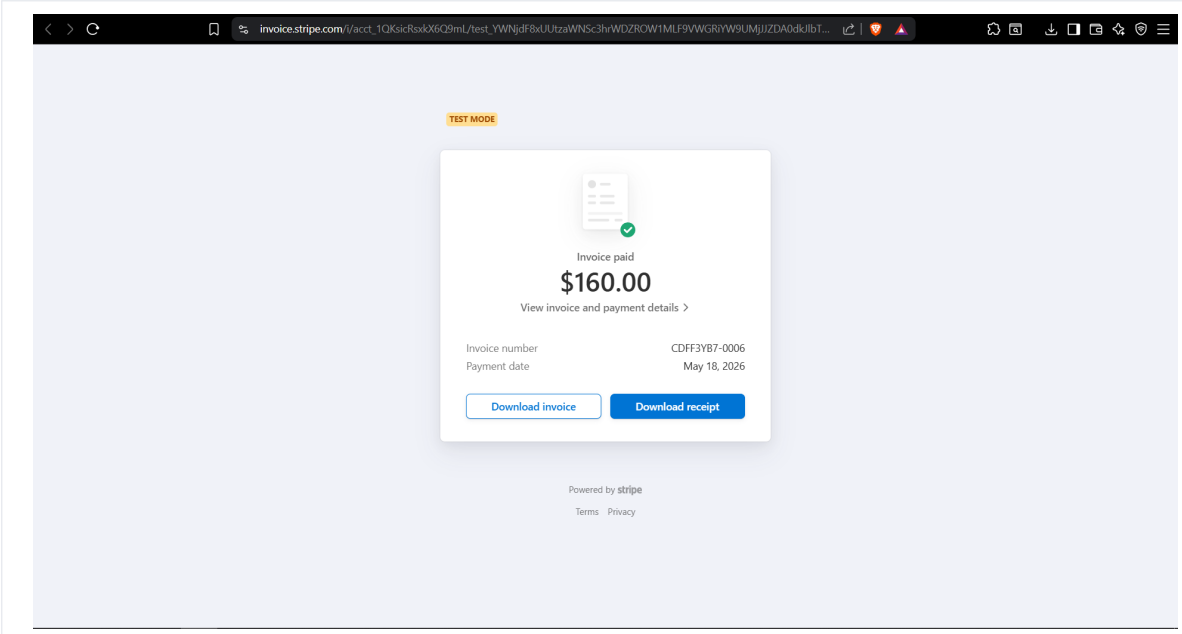

Figure B.12: Invoice – Appendix View

tration page, the Stripe checkout redirect screen, the invoice Portable Document Format view, the campaign email statistics panel, and the support chatbot panel.

C Application Programming Interface Endpoint Reference

Table C.1 lists all main Collectra Representational State Transfer Application Programming Interface endpoints. Endpoints under the `/public/` prefix do not require authentication and are accessible via a signed JSON Web Token embedded in the Uniform Resource Locator.

The endpoints are grouped below by domain to mirror the backend route modules in `apps/api`. Authentication routes establish the session cookie; workspace routes enforce tenant boundaries; campaign and debt routes support agent operations; public routes serve customers through signed tokens only.

C.0.1 Authentication and Workspace Endpoints

These routes manage registration, login, workspace provisioning, invitations, and membership. All authenticated calls expect a valid Supabase-issued JSON Web Token carried in a Hypertext Transfer Protocol-only cookie.

C.0.2 Campaign, Debt, and Public Customer Endpoints

Campaign routes cover Create, Read, Update, Delete operations, Comma-Separated Values import, and email statistics. Debt routes expose lifecycle updates, personal link generation, and promise recording. Public routes never trust client-supplied workspace identifiers: the signed token is the sole authorization mechanism for customer-facing actions.

C.0.3 Representational State Transfer Endpoint Catalogue

Method	Path	Auth	Description
POST	`/auth/register`	No	Register a new user
POST	`/auth/login`	No	Login, receive session cookie
POST	`/auth/logout`	Yes	Logout and clear session
POST	`/workspaces`	Yes	Create a workspace
GET	`/workspaces/:id`	Yes	Get workspace details
POST	`/workspaces/:id/invitations`	Yes (Manager)	Invite a team member
PATCH	`/workspaces/:id/members/:uid`	Yes (Manager)	Update a member's role
POST	`/invitations/:token/accept`	No	Accept an invitation
POST	`/workspaces/:id/campaigns`	Yes	Create a campaign
GET	`/workspaces/:id/campaigns`	Yes	List campaigns
PATCH	`/campaigns/:id`	Yes (Manager)	Update campaign settings
DELETE	`/campaigns/:id`	Yes (Manager)	Delete a campaign
POST	`/campaigns/:id/import`	Yes	Import debts via Comma-Separated Values

Method	Path	Auth	Description
GET	/campaigns/:id/email-stats	Yes	Get campaign email tracking statistics
POST	/campaigns/:id/brevo-logs/sync	Yes	Synchronize Brevo provider events
GET	/campaigns/:id/debts	Yes	List debts with filters
GET	/debts/:id	Yes	Get debt details
PATCH	/debts/:id/status	Yes	Update debt status
POST	/debts/:id/links	Yes	Generate customer personal link
GET	/debts/:id/links	Yes	List links for a debt
GET	/public/debts/:token	No	Customer views debt via secure token
POST	/public/debts/:token/promise	No	Customer submits promise-to-pay date
POST	/public/debts/:token/demo-payment	No	Confirm demo payment (development only)†
POST	/public/debts/:token/stripe/checkout-session	No	Create Stripe Checkout session
GET	/public/debts/:token/verify-payment	No	Verify Stripe payment after redirect
GET	/public/debts/:token/invoice	No	Open hosted invoice / printable Portable Document Format
GET	/public/debts/:token/track-click	No	Track customer click and redirect
POST	/public/debts/:token/track-open	No	Track customer page open event
GET	/public/debts/:debtId/open.gif	No	Email open tracking pixel endpoint
POST	/debts/:id/promises	Yes	Record payment promise
GET	/workspaces/:id/history	Yes	Get workspace action history
GET	/workspaces/:id/dashboard	Yes (Manager)	Get dashboard indicators

Table C.1: Collectra Application Programming Interface Endpoint Reference

† The `/demo-payment` endpoint simulates payment confirmation in the development and staging environment. It is disabled and not reachable in the production deployment (`collectra.xyz`).

C.0.4 Standard Error Response Format

When validation or authorization fails, the Application Programming Interface returns a JSON object with a stable `code`, a human-readable `message`, and optional `details` for field-level errors (notably during Comma-Separated Values import). Table C.2 summarises the most frequent codes observed during sprint testing.

Front-end forms map these codes to inline messages so agents can correct inputs without reading server logs. During the internship, maintaining a shared error catalogue in Linear reduced duplicate bug reports when the same validation rule appeared on both the import dialog and the debt status modal.

Code	HTTP	Typical Cause
UNAUTHORIZED	401	Missing or expired session cookie
FORBIDDEN	403	Authenticated but wrong workspace or role
NOT_FOUND	404	Unknown campaign, debt, or invitation token
VALIDATION_ERROR	400	Zod schema rejection on request body
INVALID_TRANSITION	409	Illegal debt status change
TOKEN_EXPIRED	410	Customer personal link past expiration
IMPORT_PARTIAL	207	Comma-Separated Values processed with skipped rows (summary returned)

Table C.2: Common Collectra Application Programming Interface Error Codes

D Database Schema

Figure D.1 shows the Entity-Relationship diagram for the Collectra database.

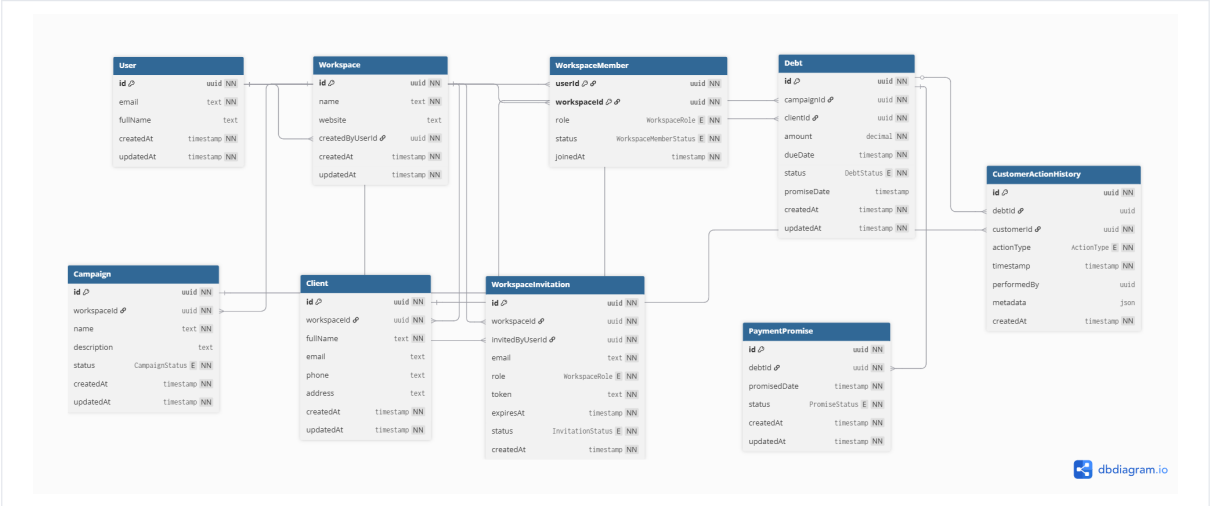

Figure D.1: Collectra Database Schema – Entity-Relationship Diagram

The main tables and their roles are:

- `User` – authenticated user accounts managed through Supabase Auth;
- `Workspace` – isolated tenant environments, each with its own data;
- `WorkspaceMember` – junction table linking users to workspaces with an assigned role;
- `Invitation` – pending or accepted membership invitations;
- `Campaign` – debt collection campaigns scoped to a workspace;
- `Debt` – individual debt records linked to a campaign and debtor;
- `PaymentPromise` – recorded payment commitment dates for debts;
- `CustomerActionHistory` – timeline of operational actions (status updates, promises, payments, tracking events);
- `BrevoEventLog` – provider-level email event logs used for diagnostics and synchronization.

D.0.1 Entity Attribute Reference

Table D.1 and Table D.2 document the principal attributes stored in PostgreSQL. Foreign keys enforce referential integrity at the database layer; Prisma migrations version every schema change applied during the internship.

Entity / Field	Description
Workspace.id	Universally unique identifier; root of tenant isolation
Workspace.name	Display name chosen at creation; unique per platform policy
WorkspaceMember.role	MANAGER or AGENT; drives middleware checks
Invitation.token	Signed token embedded in acceptance email link
Invitation.status	PENDING, ACCEPTED, or EXPIRED
Campaign.workspaceId	Foreign key; cascades logically with workspace deletion rules
Campaign.title	Human-readable label shown in agent navigation

Table D.1: Workspace and Campaign Entities – Key Attributes

Entity / Field	Description
Debt.amount	Decimal amount owed; validated on import and public view
Debt.currency	ISO 4217 code (default usd); passed to Stripe Checkout and invoices
Debt.status	Lifecycle state (see Table 2.7)
Debt.dueDate	Used for overdue detection and dashboard filters
Debt.customerEmail	Contact channel; deduplicated within campaign on import
PaymentPromise.promisedAt	Date committed by customer or agent
CustomerActionHistory.actionType	Normalised event type for audit timeline
CustomerActionHistory.metadata	JSON payload (previous status, payment session id, etc.)
BrevoEventLog.eventType	Provider event (EMAIL_SENT, LINK_CLICKED, …)

Table D.2: Debt, Promise, History, and Tracking Entities – Key Attributes

D.0.2 Indexing and Query Patterns

Dashboard queries aggregate counts grouped by `status` and `campaignId`. Imports batch-insert debts inside transactions to keep partial failures recoverable. Personal links index `token` hashes for constant-time lookup on public routes. These choices were validated on preview deployments with datasets exceeding five thousand debts per workspace.